

浙江大学

运筹学大作业报告



攻击视觉大模型！

Attacking Vision-Language Models!

学 生 姓 名： 张天羽 江征远 金震庭 田 阔

专 业 名 称： 自动化（控制）

所 在 学 院： 控制科学与工程学院

二〇二六年六月二十四日

摘要

想象：敌人正在使用视觉大模型，例如敌方轰炸机正在识别目标，又如敌方总统府正在安检，再如敌方医疗模型正在识别肿瘤——在这些场景下，视觉大模型一般不会独立工作，而会有一个敌方人类检查员，检查输入图片 x 的质量。此时，我方能否通过“篡改输入图片 x ”的方式，将输入图片 x 篡改为 x' ，使得敌方人类检查员无法发现 x' 与 x 的区别，但却能使得敌方模型分类错误呢？

这就是“攻击视觉大模型”。现有研究大多基于随机梯度、启发式搜索或深度学习经验策略，而较少从传统优化理论出发，这使得许多攻击方法虽然在实验上有效，但缺乏可解释性与理论保证。基于此，我们尝试从运筹学视角出发，将对抗攻击问题形式化为约束优化问题，并尽可能采用课程中学习的经典方法进行求解与分析。

首先，我们将攻击问题建模成一个有约束的非线性规划问题。

然后，我们采用非迭代法，包括 KKT 条件与 Lagrange 函数法求解。

随后，我们采用迭代法，包括最速下降法、牛顿法、拟牛顿法、共轭梯度法与外点法求解。

随后，我们将原问题化为对偶问题并求解，同时验证了弱对偶性、最优性与强对偶性。

此外，我们发现只要合理设计目标函数，就能让模型产生指定的错误，例如厕纸。

最后，我们指定一种特殊形状的扰动（“张天羽”三字），采用反向贪心剪枝求解。

在对于攻击效果与扰动强度的优化效果上，化为对偶问题并求解的效果最好，其次是非迭代法与外点法；所有下降迭代法因为只能处理软约束，所以对扰动强度的优化效果不好。

在攻击效率上，所有只依赖梯度的方法远远快于依赖 Hessian 矩阵的方法，并且攻击效果并未因缺少二阶信息而明显减弱；反向贪心剪枝效率提升明显，远远快于枚举。

此外，我们特意对比了手搓求解器与开源优化器的效果与效率。因为手搓求解器对于特定问题更有针对性，所以效果与效率甚至比开源优化器更好。

关键词：视觉分类模型；非线性规划；对偶问题；图搜索

ABSTRACT

Imagine that the enemy is using a visual large model: for example, an enemy bomber is identifying targets; or an enemy presidential residence is conducting security checks; or an enemy medical model is identifying tumors. In these scenarios, a visual large model generally does not work independently; instead, there is an enemy human inspector who checks the quality of the input image x . At this point, can our side, by “tampering with the input image x ,” tamper with the input image x into x' , so that the enemy human inspector cannot detect the difference between x' and x , but the enemy model misclassifies it?

This is “attacking visual large models.” Existing research is mostly based on random gradients, heuristic search, or deep-learning empirical strategies, and less often starts from traditional optimization theory. This makes many attack methods effective in experiments, but lacking in interpretability and theoretical guarantees. Based on this, we attempt to start from the perspective of operations research, formalize the adversarial attack problem as a constrained optimization problem, and solve and analyze it as much as possible using the classical methods learned in the course.

First, we model the attack problem as a constrained nonlinear programming problem.

Then, we adopt non-iterative methods, including KKT conditions and the Lagrange function method, to solve it.

Subsequently, we adopt iterative methods, including the steepest descent method, Newton’s method, quasi-Newton method, conjugate gradient method, and exterior point method, to solve it.

After that, we transform the primal problem into a dual problem and solve it, while verifying weak duality, optimality, and strong duality.

In addition, we find that as long as the objective function is reasonably designed, the model can be made to produce a specified error, such as toilet paper.

Finally, we specify a perturbation of a special shape — the three Chinese characters “张天羽” — and solve it using reverse greedy pruning.

In terms of the optimization effects on attack effectiveness and perturbation strength, transforming the problem into a dual problem and solving it achieves the best result, followed by non-iterative methods and the exterior point method; all descent iterative methods can only handle soft constraints, so their optimization effects on perturbation strength are poor.

In terms of attack efficiency, all methods that rely only on gradients are far faster than methods that rely on Hessian matrices, and the attack effectiveness is not significantly weakened by the lack of second-order information; reverse greedy pruning significantly improves efficiency and is far faster than enumeration.

In addition, we deliberately compare the effectiveness and efficiency of hand-written solvers and open-source optimizers. Because hand-written solvers are more targeted for specific problems, their effectiveness and efficiency are even better than those of open-source optimizers.

KEY WORDS: visual classification model; nonlinear programming; dual problem; graph search

致 谢

转眼已是夏学期的尾声，感谢何老师一个学期以来的辛苦付出。在过去的两个月里，我们常常被何老师的热情与责任心打动，也常常担心自己没有^在运筹学上投入足够多的精力。我们组的几位同学偶尔因为熬夜准备比赛，早八没起床上课，让何老师操心了，我们在此表达深深的歉意！

但让我们自信的是：我们在大作业上投入了无人能及的热情与心血。我们精心选取模型、搭建数据集、推导证明、设计实验；我们的报告几乎没有使用 AI，完全是对照课件，一字一句手写的。等到我们写完报告，推导过程已经倒背如流，实在是呕心沥血了！

在 PPT 分享环节，因为我们表达能力有限，也许没有让大家听懂基本原理。后来，我们针对三个最基本的问题 —— “什么是图片”“什么是视觉分类模型”“什么是攻击”，手绘了通俗易懂的示意图，然后使用 GPT image 2 润色图片。这三张图片真的是包教包会了，相信读者一定能理解基本原理！

为了方便何老师速读，我们将关键字句涂成了红色。只看这些段落不会影响行文思路。此外，我们的模型、数据集、源代码、实验结果已经全部开源到 Github，网址如下：

<https://github.com/LocHlaith/Attacking-Vision-Language-Models>

目 录

摘 要	I
ABSTRACT	II
致 谢	III
目 录	IV
1 前言	1
1.1 选题背景	1
1.2 研究现状	1
1.3 选题意义	2
2 问题描述	3
2.1 什么是图片？	3
2.2 什么是视觉分类模型？	3
2.3 什么是攻击？	4
3 非迭代法	6
3.1 不等式约束问题 —— KKT 条件	6
3.1.1 方法设计	6
3.1.2 实验结果	6
3.2 等式约束问题 —— Lagrange 函数法	8
3.2.1 方法设计	8
3.2.2 实验结果	10
4 迭代法	12
4.1 化为无约束问题	12
4.1.1 加权法	12
4.1.2 外点法	12
4.1.3 投影法	12
4.2 最速下降法	13
4.2.1 方法设计	13
4.2.2 实验结果	13
4.3 牛顿法	15
4.3.1 方法设计	15
4.3.2 实验结果	16
4.4 拟牛顿法	19
4.4.1 方法设计	19

4.4.2 实验结果	20
4.5 共轭梯度法	23
4.5.1 方法设计	23
4.5.2 实验结果	24
4.6 外点法模型	27
4.6.1 方法设计	27
4.6.2 实验结果	29
4.7 投影法模型	30
4.7.1 方法设计	30
4.7.2 实验结果	31
5 化为对偶问题	33
5.1 方法设计	33
5.1.1 弱对偶性分析	33
5.1.2 最优性分析	34
5.1.3 强对偶性分析	34
5.1.4 互补松弛性分析	35
5.1.5 方法设计	35
5.2 实验结果	37
6 让模型将任何东西识别为厕纸	39
6.1 实验结果	40
7 面积最小的“张天羽”	42
7.1 建立模型	42
7.1.1 图像网格图	42
7.1.2 字形模板	42
7.1.3 0-1 非线性规划模型	43
7.1.4 基于网络流的连通性模型	44
7.1.5 基于一阶近似的混合整数线性规划模型	44
7.2 方法设计	45
7.3 实验结果	46
7.3.1 反向贪心剪枝	46
7.3.2 一阶线性化 MILP	47
8 小组分工	49
参考文献	50
附 录	53

1 前言

1.1 选题背景

近年来，深度学习在计算机视觉领域取得了突破性进展，卷积神经网络（CNN）在图像分类、目标检测与语义分割等任务中表现出接近甚至超过人类水平的性能。然而，Szegedy 等人首次发现，**深度神经网络对输入的微小扰动具有高度敏感性，即所谓“对抗样本”现象^[1]**。随后 Goodfellow 等人进一步指出，这类扰动可以通过简单的梯度方向构造，从而系统性地欺骗模型^[2]。

随着研究深入，研究者发现对抗样本不仅存在于理论实验中，还具有广泛的工程威胁。例如，通过快速梯度方法（FGSM）可以在单步计算内生成攻击样本^[2]，而基于迭代优化的方法（如 PGD）则能够在约束优化框架下显著增强攻击效果^[9]。此外，Moosavi-Dezfooli 等提出的 DeepFool 方法从最小扰动范数角度刻画攻击问题，本质上构成一个几何优化问题^[3]。

在视觉模型方面，MobileNetV2 作为轻量级神经网络结构，被广泛部署于移动端与嵌入式设备中，其在计算资源受限条件下仍保持较高精度，因此成为对抗攻击研究的重要目标模型之一。与此同时，已有研究表明对抗攻击在不同网络结构之间具有良好的迁移性，即在一个模型上生成的扰动可以欺骗其他模型^[27]。

此外，对抗攻击已从数字空间扩展到物理世界，例如对真实交通标志、摄像头系统的攻击验证了其现实威胁性^[10]。因此，研究视觉模型（尤其是 MobileNetV2）的攻击机制具有重要理论与实践意义。

1.2 研究现状

当前对抗攻击研究主要围绕攻击方法建模、优化求解与跨模型迁移性三个方向展开。

在攻击方法方面，研究者提出了多种基于优化的攻击策略。FGSM 通过一次梯度符号方向扰动实现快速攻击^[2]；DeepFool 则通过迭代线性化模型寻找最小扰动边界^[3]；Carlini & Wagner 攻击进一步将对抗样本生成问题转化为带约束的优化问题，并通过精细化目标函数提升攻击强度^[7]。

在黑盒攻击领域，研究重点在于在不可访问梯度的情况下构造对抗样本。例如，基于零阶优化的 ZOO 方法^[28]以及基于决策边界查询的攻击方法^[29]，均将攻击问题转化为数值优化问题。同时，Ilyas 等人证明有限查询条件下仍可通过梯度估计实现有效攻击^[30]。

在迁移性研究方面，研究表明对抗样本在不同模型之间具有显著迁移能力。Dong 等提出动量迭代方法增强攻击稳定性^[13]，Xie 等通过输入多样性提升迁移效果^[12]，进一步说明对抗扰动具有跨模型优化一致性特征。

在防御方向上，Madry 等提出的对抗训练将问题建模为 min-max 优化问题^[9]，成为后续鲁棒优化研究基础；TRADES 方法则从理论角度刻画鲁棒性与精度之间的权衡关系^[16]。

此外，近年来 Vision Transformer 等新架构也被证明同样易受攻击^{[24][25]}，说明对抗攻击问题具有模型无关性。

总体来看，现有研究已从启发式方法逐渐转向优化视角，但仍存在一定局限：例如缺乏严格数学证明、对优化结构刻画不足，以及较少利用经典运筹学工具进行统一建模。

1.3 选题意义

现有研究大多基于随机梯度、启发式搜索或深度学习经验策略，而较少从传统优化理论出发，例如凸优化思想、对偶问题思想或图搜索等方法进行严格建模与证明^{[7][28][29]}。这使得许多攻击方法虽然在实验上有效，但缺乏可解释性与理论保证。

基于此，我们尝试从运筹学视角出发，将对抗攻击问题形式化为约束优化问题，并尽可能采用课程中学习的经典方法进行求解与分析。同时，我们在推导过程中，注重数学严格性与可证明性，从而弥补现有研究中“强实验、弱理论”的不足。

因此，我们选题不仅具有人工智能安全领域的重要应用价值，同时也体现了运筹学方法在现代机器学习问题中的扩展能力，有助于加强优化理论与深度学习之间的交叉融合。

2 问题描述

2.1 什么是图片？

视觉大模型一般对输入图片的尺寸有特定的要求。如果输入图片的尺寸不满足要求，必须强制缩放到要求尺寸，才能输入模型。对于 MobileNetV2 来说，则要求输入图片的尺寸为 $224 \times 224 \text{px}^2$ 。每个像素点的颜色是由红、绿、蓝（RGB）3 种颜色合成的，故每个像素点对应 3 个参数。综上，整张图片就是一个 $224 \times 224 \times 3 = 150528$ 维的向量：

$$\mathbf{x} = (x_1, x_2, \dots, x_{150528})^T \in \mathbb{R}^{150528}. \quad (2-1)$$

图 2-1 是图片向量的示意图，图中散落的方块表示像素点。为了避免看不清，没有完整画出 224×224 个像素点。左上角展示了颜色是如何由红、绿、蓝 3 种颜色合成的。

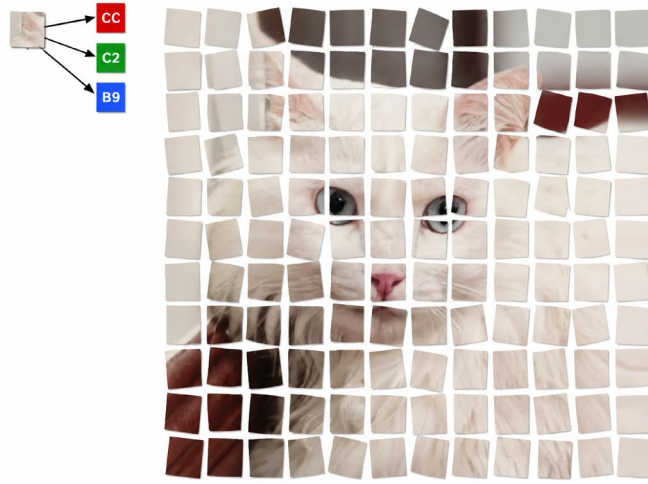


图 2-1 图片是一个 $224 \times 224 \times 3 = 150528$ 维的向量。

图中散落的方块表示像素点。左上角展示了颜色是如何由红、绿、蓝 3 种颜色合成的。

2.2 什么是视觉分类模型？

视觉分类模型能、且只能将输入图片归类为自带类别中的一类，而不能将输入图片归类为自带类别外的其他类别。对于 MobileNetV2 来说，自带类别为 ImageNet-1K 即 1000 类，对于每一类，都有一个得分函数 $z_j(\mathbf{x})$ ，例如

$$\begin{aligned} z_{817}(\mathbf{x}) &= z_{\text{Are you a sports car?}}(\mathbf{x}), \\ z_{283}(\mathbf{x}) &= z_{\text{Are you a Persian cat?}}(\mathbf{x}), \\ z_{701}(\mathbf{x}) &= z_{\text{Are you a Windsor tie?}}(\mathbf{x}), \\ z_{985}(\mathbf{x}) &= z_{\text{Are you a daisy?}}(\mathbf{x}), \\ &\dots \end{aligned} \quad (2-2)$$

对于输入图片 $\mathbf{x}$ ，模型计算出每一类的得分函数 $z_1(\mathbf{x}), z_2(\mathbf{x}), \dots, z_{1000}(\mathbf{x})$ ，然后取

$$\widehat{\text{balabala}}(\mathbf{x}) = \arg\{\max[z_{\text{Are you a balabala?}}(\mathbf{x})]\}, \quad (2-3)$$

$\widehat{\text{balabala}}$ 即为模型所认为的这张图片应归属的类。

图 2-2 是视觉分类模型决策过程的示意图。为了避免看不清，没有画出完整的 1000 类。下方的“温度计”表示各个类别的得分函数。

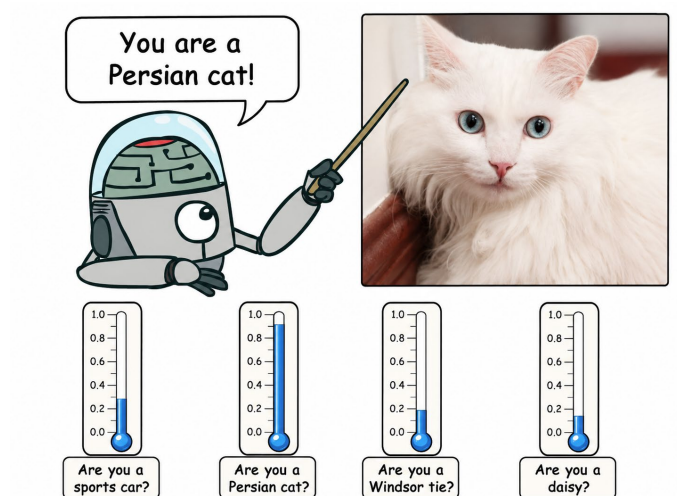


图 2-2 视觉分类模型的决策过程。下方的“温度计”表示各个类别的得分函数。

2.3 什么是攻击？

想象：敌人正在使用视觉大模型，例如敌方轰炸机正在识别目标，又如敌方总统府正在安检，再如敌方医疗模型正在识别肿瘤——在这些场景下，视觉大模型一般不会独立工作，而会有一个敌方人类检查员，检查输入图片 x 的质量。

此时，我方尝试通过“篡改输入图片 x ”的方式，将输入图片 x 篡改改为 x' ，使得敌方人类检查员无法发现 x' 与 x 的区别，但却能使得敌方模型分类错误！

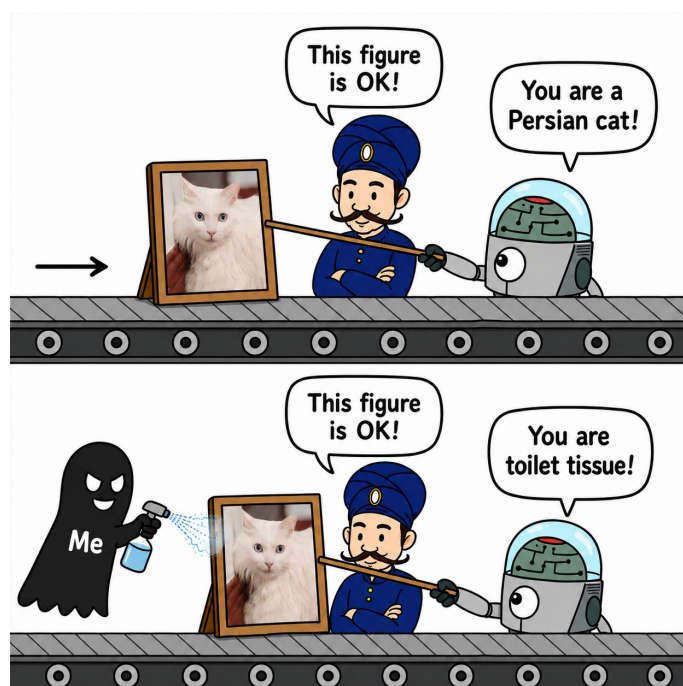


图 2-3 我方尝试通过“篡改输入图片 x ”的方式，将输入图片 x 篡改改为 x' ，使得敌方人类检查员无法发现 x' 与 x 的区别，但却能使得敌方模型分类错误。

下面对攻击问题进行建模：

设被我方篡改后的输入图像为 x' ，欲使敌方模型分类错误，只需使

$$\begin{aligned} \widehat{\text{balabala}}'(x') &= \arg\{\max[z_{\text{Are you a balabala?}}(x')]\} \\ &\neq \widehat{\text{balabala}}(x) = \arg\{\max[z_{\text{Are you a balabala?}}(x)]\}, \end{aligned} \quad (2-4)$$

即 $z_{\text{Are you a balabala?}}(x')$ 不再是各个类别的得分函数的最大值。然而 $\arg\{\max[\cdot]\}$ 函数较难处理，因此我们不难想到：让 $z_{\text{Are you a balabala?}}(x')$ 尽可能小，就能尽可能让 $z_{\text{Are you a balabala?}}(x')$ 不再是各个类别的得分函数的最大值。则问题化为：

$$\min z_{\text{Are you a balabala?}}(x') \quad (2-5)$$

然而，得分函数 $z_j(x)$ 是由 $x_1, x_2, \dots, x_{150528}$ 经卷积、批归一化、ReLU6 激活、深度可分离卷积、倒残差结构、全局平均池化和线性分类层得到的，因此得分函数 $z_j(x)$ 不可导。实际上，前人一般令目标函数为：

$$\min \Phi(x, \widehat{\text{balabala}}) = z_{\text{Are you a balabala?}}(x) - \log \left[\sum_{j=1}^{1000} e^{z_j(x)} \right]. \quad (2-6)$$

为了使敌方人类检查员无法发现 x' 与 x 的区别，设扰动强度 $\delta = x' - x$ ，理论上，只需限制 δ 的任意一种范数，就能限制 δ 。因为 $\|\delta\|_2^2$ 可导，且为凸函数，所以我们不妨选取 $\|\delta\|_2^2$ 作为约束，即

$$\begin{aligned} \min_{\delta \in \mathbb{R}^n} \quad & \Phi(\delta) = z_{\text{Are you a balabala?}}(x + \delta) - \log \left[\sum_{j=1}^{1000} e^{z_j(x+\delta)} \right], \\ \text{s.t.} \quad & \|\delta\|_2^2 \leq \varepsilon^2. \end{aligned} \quad (2-7)$$

即

$$\begin{aligned} \min_{\delta \in \mathbb{R}^n} \quad & \Phi(\delta) = z_{\text{Are you a balabala?}}(x + \delta) - \log \left[\sum_{j=1}^{1000} e^{z_j(x+\delta)} \right], \\ \text{s.t.} \quad & g(\delta) = \varepsilon^2 - \|\delta\|_2^2 \geq 0. \end{aligned} \quad (2-8)$$

综上，我们将攻击问题化为了一个有约束的非线性规划问题。我们惊喜地发现 $g(\delta)$ 是一个凹函数！但可惜目标函数是关于 $x_1, x_2, \dots, x_{150528}$ 的不可导的非线性函数，不知道是不是凸函数，因此不一定能找到全局最优解。

3 非迭代法

我们原本称这一章为“解析法”，但严格来说，对目标函数 $\Phi(\delta)$ 无法做解析操作、只能做数值操作，因此我们称这一章为“非迭代法”，依次尝试了 KKT 条件与 Lagrange 函数法。

3.1 不等式约束问题 —— KKT 条件

3.1.1 方法设计

问题

$$\begin{aligned} \min_{\delta \in \mathbb{R}^n} \quad & \Phi(\delta) = z_{\text{Are you a balabala?}}(x + \delta) - \log \left[\sum_{j=1}^{1000} e^{z_j(x+\delta)} \right] \\ \text{s.t.} \quad & g(\delta) = \varepsilon^2 - \|\delta\|_2^2 \geq 0. \end{aligned} \quad (2-8)$$

是一个不等式约束问题，因此尝试采用 KKT 条件求解。

驻点条件：

$$\nabla \Phi(\delta^*) - \mu^* \nabla g(\delta^*) = 0. \quad (3-1)$$

互补松弛条件：

$$\mu^* g(\delta^*) = 0. \quad (3-2)$$

非负条件：

$$\mu^* \geq 0. \quad (3-3)$$

若 $g(\delta) = \varepsilon^2 - \|\delta\|_2^2 = 0$ ，则问题 2-8 不再是不等式约束问题，这不是本小节所要讨论的。因此，我们令 $g(\delta) = \varepsilon^2 - \|\delta\|_2^2 > 0$ ，根据互补松弛条件，有 $\mu^* = 0$ ，则驻点条件化为：

$$\nabla \Phi(\delta^*) = 0. \quad (3-4)$$

注意到 $\Phi(\delta) = \Phi(0 + \delta)$ ，对其进行二阶泰勒展开，则有：

$$\Phi(\delta) \approx \Phi(0) + [\nabla_x \Phi(0)]^\top \delta + \frac{1}{2} \delta^\top [\nabla_x^2 \Phi(0)] \delta. \quad (3-5)$$

等式两边对 δ 求导，则有：

$$\nabla \Phi(\delta) \approx \nabla_x \Phi(0) + [\nabla_x^2 \Phi(0)] \delta. \quad (3-6)$$

代入驻点条件 3-4，则有：

$$\nabla \Phi(\delta^*) = \nabla_x \Phi(0) + [\nabla_x^2 \Phi(0)] \delta^* = 0, \quad (3-7)$$

解得

$$\delta^* = -[\nabla_x^2 \Phi(0)]^{-1} \nabla_x \Phi(0), \quad (3-8)$$

即为局部最优解。因为我们只能保证 $g(\delta)$ 为凹函数，而不知道 $\Phi(\delta)$ 是不是凸函数，因此不能保证 δ^* 是全局最优解。

3.1.2 实验结果

我们统一选取 4 张原图生成攻击图，四张原图如图 3-1 所示，其中，sports car 与 Persian cat 这两张图片的共同特征是主体明确、单一；daisy 图片的主体同样明确，但数量众多；Windsortie 图片主体不明确，可能被人类理解为原始人、西装客，或者其他类别。这样就可以验证攻击方法在不同

风格的图片上的攻击效果。



(a) 原图: sports car

14.517%

(b) 原图: Persian cat

9.729%



(c) 原图: Windsor tie

12.536%

(d) 原图: daisy

13.113%

图 3-1 我们统一选取的 4 张原图

本文所有实验都有 2 套版本: 在 **manual** 版本中, 我们尝试了手搓求解器; 在 **efficient** 版本中, 我们调用现有的求解器 (例如 Adam 优化器), 以对比手搓求解器与开源求解器的效果与效率。



(a) 原图: sports car

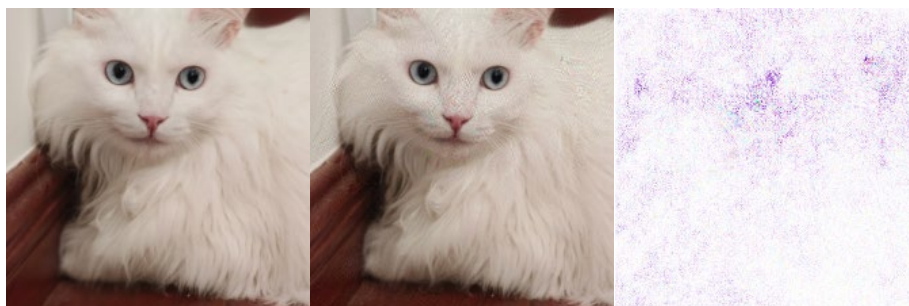
14.517%

(a1) 攻击图: convertible

7.2838%

(a2) 扰动图

$L_2 = 3.8161$



(b) 原图: Persian cat

(b1) 攻击图: paper towel

(b2) 扰动图



图 3-2 不等式约束问题 —— KKT 条件实验结果

表 3-1 不等式约束问题 —— KKT 条件实验结果

求解器	成功率	平均耗时/s	最大耗时/s	平均 $\Phi'(\delta^*)/\%$	平均 L_2
manual	4/4	56.442	69.630	8.269	3.923
efficient	4/4	55.922	66.698	8.269	3.923

可见，针对 4 张图片的攻击全部成功，且肉眼不易发现攻击图与原图的区别。扰动图几乎为白色，说明所需的扰动强度很小。

然而，我们发现 L_2 有一个特例，即图 3-2 d2，其 $L_2 = 4.0000 = \varepsilon^2$ ，说明约束 $g(\delta) = \varepsilon^2 - L_2 \geq 0$ 起作用，这与本小节的假设不符。这是因为我们在式 3-4 中令 $\mu = 0$ ，这不是 $g(\delta) = \varepsilon^2 - L_2 > 0$ 的充分条件，不能保证约束 $g(\delta) = \varepsilon^2 - L_2 \geq 0$ 不起作用。

攻击每张图片的耗时约为 1 分钟。经过分步测量，发现 Hessian 矩阵 $\nabla_x^2 \Phi(0)$ 的存储与求逆的耗时最大，这是因为本问题的 Hessian 矩阵是一个 150528×150528 的矩阵，维数巨大。

3.2 等式约束问题 —— Lagrange 函数法

3.2.1 方法设计

对于问题

$$\begin{aligned} \min_{\delta \in \mathbb{R}^n} \quad & \Phi(\delta) = z_{\text{Are you a balabala?}}(x + \delta) - \log \left[\sum_{j=1}^{1000} e^{z_j(x+\delta)} \right], \\ \text{s.t.} \quad & g(\delta) = \varepsilon^2 - \|\delta\|_2^2 \geq 0. \end{aligned} \quad (2-8)$$

为了让目标函数 $\Phi(\delta)$ 尽可能小，直觉上，极小值很可能在扰动强度 $\|\delta\|_2^2$ 最大时取到，即

$$\|\delta\|_2^2 = \varepsilon^2, \quad (3-9)$$

此时，式 2-8 化为等式约束问题：

$$\begin{aligned} \min_{\delta \in \mathbb{R}^n} \quad & \Phi(\delta) = z_{\text{Are you a balabala?}}(x + \delta) - \log \left[\sum_{j=1}^{1000} e^{z_j(x+\delta)} \right], \\ \text{s.t.} \quad & g(\delta) = \varepsilon^2 - \|\delta\|_2^2 = 0. \end{aligned} \quad (3-10)$$

因此尝试采用 Lagrange 函数法求解。

定义 Lagrange 函数：

$$L(\delta, \lambda) = \Phi(\delta) - \lambda g(\delta). \quad (3-11)$$

则有：

$$\begin{cases} \left. \frac{\partial L(\delta, \lambda)}{\partial \delta} \right|_{\delta^*} = \nabla \Phi(\delta^*) - \lambda \nabla g(\delta^*) = 0; \\ \left. \frac{\partial L(\delta, \lambda)}{\partial \lambda} \right|_{\lambda^*} = -g(\delta^*) = 0. \end{cases} \quad (3-12)$$

我们在 3.1.1 节已经证明：

$$\nabla \Phi(\delta) \approx \nabla_x \Phi(0) + [\nabla_x^2 \Phi(0)] \delta. \quad (3-6)$$

代入式 3-12，则有：

$$\begin{cases} \left. \frac{\partial L(\delta, \lambda)}{\partial \delta} \right|_{\delta^*} = \nabla_x \Phi(0) + [\nabla_x^2 \Phi(0)] \delta^* - \lambda \nabla g(\delta^*) = 0; \\ \left. \frac{\partial L(\delta, \lambda)}{\partial \lambda} \right|_{\lambda^*} = -g(\delta^*) = 0. \end{cases} \quad (3-13)$$

即

$$\begin{cases} \nabla_x \Phi(0) + [\nabla_x^2 \Phi(0)] \delta^* + 2\lambda \delta^* = 0; \\ \delta^{*\top} \delta^* - \varepsilon^2 = 0. \end{cases} \quad (3-14)$$

解得

$$\delta^* = -[\nabla_x^2 \Phi(0) + 2\lambda I]^{-1} \nabla_x \Phi(0) \approx -\frac{1}{2\lambda} \nabla_x \Phi(0). \quad (3-15)$$

代入式 3-14，则有：

$$\frac{1}{4\lambda^2} [\nabla_x \Phi(0)]^\top [\nabla_x \Phi(0)] - \varepsilon^2 = 0, \quad (3-16)$$

解得

$$\lambda^* = \frac{1}{2\varepsilon} \sqrt{[\nabla_x \Phi(0)]^\top [\nabla_x \Phi(0)]}. \quad (3-17)$$

代入式 3-15，则有：

$$\delta^* = -\varepsilon \frac{\nabla_x \Phi(0)}{\sqrt{[\nabla_x \Phi(0)]^\top [\nabla_x \Phi(0)]}}, \quad (3-18)$$

即为局部最优解。同理，因为不知道 $\Phi(\delta)$ 是不是凸函数，所以不能保证 δ^* 是全局最优解。

3.2.2 实验结果



图 3-3 等式约束问题 —— Lagrange 函数法实验结果

表 3-2 等式约束问题 —— Lagrange 函数法实验结果

求解器	成功率	平均耗时/s	最大耗时/s	平均 $\Phi'(\delta^*)/\%$	平均 L_2
manual	4/4	0.079	0.201	8.212	4.000
efficient	4/4	0.070	0.164	8.212	4.000

可见，针对 4 张图片的攻击全部成功，且肉眼不易发现攻击图与原图的区别。扰动图几乎为白色，说明所需的扰动强度很小。

因为本小节讨论等式约束，所以可以发现所有扰动强度 $L_2 = 4.0000$ ，严格满足等式约束。

攻击每张图片的耗时约为 0.1 秒，约为 KKT 条件方法的 $\frac{1}{600}$ 。这是因为 Lagrange 函数法无需计算 Hessian 矩阵 $\nabla_x^2 \Phi(0)$ ，大大提高攻击效率。

4 迭代法

接下来，我们尝试采用下降迭代法求解，包括最速下降法、牛顿法、拟牛顿法、共轭梯度法。

4.1 化为无约束问题

下降迭代法只能处理无约束问题。因此，必须首先将问题 2-8 化为无约束问题。我们分别尝试采用加权法、外点法、投影法。

4.1.1 加权法

对于问题

$$\begin{aligned} \min_{\delta \in \mathbb{R}^n} \quad & \Phi(\delta) = z_{\text{Are you a balabala?}}(x + \delta) - \log \left[\sum_{j=1}^{1000} e^{z_j(x+\delta)} \right], \\ \text{s.t.} \quad & g(\delta) = \varepsilon^2 - \|\delta\|_2^2 \geq 0. \end{aligned} \quad (2-8)$$

加权法的思想类似于目标规划的思想，将 $g(\delta) = \varepsilon^2 - \|\delta\|_2^2 \geq 0$ 视为软约束，则问题 2-8 化为

$$\min Q(\delta) = \Phi(\delta) + P\|\delta\|_2^2. \quad (4-1)$$

$P > 0$ 为优先因子。当 P 较大时，优先极小化扰动强度 $\|\delta\|_2^2$ ；当 P 较小时，优先极小化 $\Phi(\delta)$ 。

4.1.2 外点法

对于问题

$$\begin{aligned} \min_{\delta \in \mathbb{R}^n} \quad & \Phi(\delta) = z_{\text{Are you a balabala?}}(x + \delta) - \log \left[\sum_{j=1}^{1000} e^{z_j(x+\delta)} \right], \\ \text{s.t.} \quad & g(\delta) = \varepsilon^2 - \|\delta\|_2^2 \geq 0. \end{aligned} \quad (2-8)$$

构造 Courant 罚函数，惩罚可行域外的迭代点：

$$\min P(\delta, M) = \Phi(\delta) + M\{\min[0, g(\delta)]\}^2. \quad (4-2)$$

$M > 0$ 为罚因子，当 M 趋向无穷时， δ^* 为问题 2-8 的最优解。

4.1.3 投影法

对于问题

$$\begin{aligned} \min_{\delta \in \mathbb{R}^n} \quad & \Phi(\delta) = z_{\text{Are you a balabala?}}(x + \delta) - \log \left[\sum_{j=1}^{1000} e^{z_j(x+\delta)} \right], \\ \text{s.t.} \quad & g(\delta) = \varepsilon^2 - \|\delta\|_2^2 \geq 0. \end{aligned} \quad (2-8)$$

投影法是前人^[7]处理扰动强度约束的常用方法。将问题 2-8 视为无约束问题进行下降迭代法求解，每迭代 1 步，检查 $g(\delta) = \varepsilon^2 - \|\delta\|_2^2 \geq 0$ 是否成立（即“投影”是否在可行域内）。直到 $g(\delta) = \varepsilon^2 - \|\delta\|_2^2 \geq 0$ 不成立，停止迭代。

4.2 最速下降法

4.2.1 方法设计

对于加权法问题

$$\min Q(\delta) = \Phi(\delta) + P\|\delta\|_2^2. \quad (4-1)$$

下降最快的方向为：

$$p^{(k)} = -\nabla Q(\delta^{(k)}). \quad (4-3)$$

取最优步长

$$\lambda_k^* = \arg \min_{\lambda} f(x^{(k)} + \lambda p^{(k)}), \quad (4-4)$$

因为

$$\frac{df(x^{(k)} + \lambda p^{(k)})}{d\lambda} = \left(\frac{\partial f(x^{(k)} + \lambda p^{(k)})}{\partial (x^{(k)} + \lambda p^{(k)})} \right)^T \frac{d(x^{(k)} + \lambda p^{(k)})}{d\lambda} = \nabla f(x^{(k+1)})^T p^{(k)} = 0, \quad (4-5)$$

所以

$$\nabla f(x^{(k+1)}) \perp p^{(k)}, \quad (4-6)$$

迭代路径呈现“之”字形，接近极值点时尤为严重。因此最速下降法只适用于迭代前期。

4.2.2 实验结果



(a) 原图：sports car
14.517%

(a1) 攻击图：pickup
26.3531%

(a2) 扰动图
 $L_2 = 7.1967$



(b) 原图：Persian cat
9.729%

(b1) 攻击图：paper towel
44.0947%

(b2) 扰动图
 $L_2 = 11.8668$

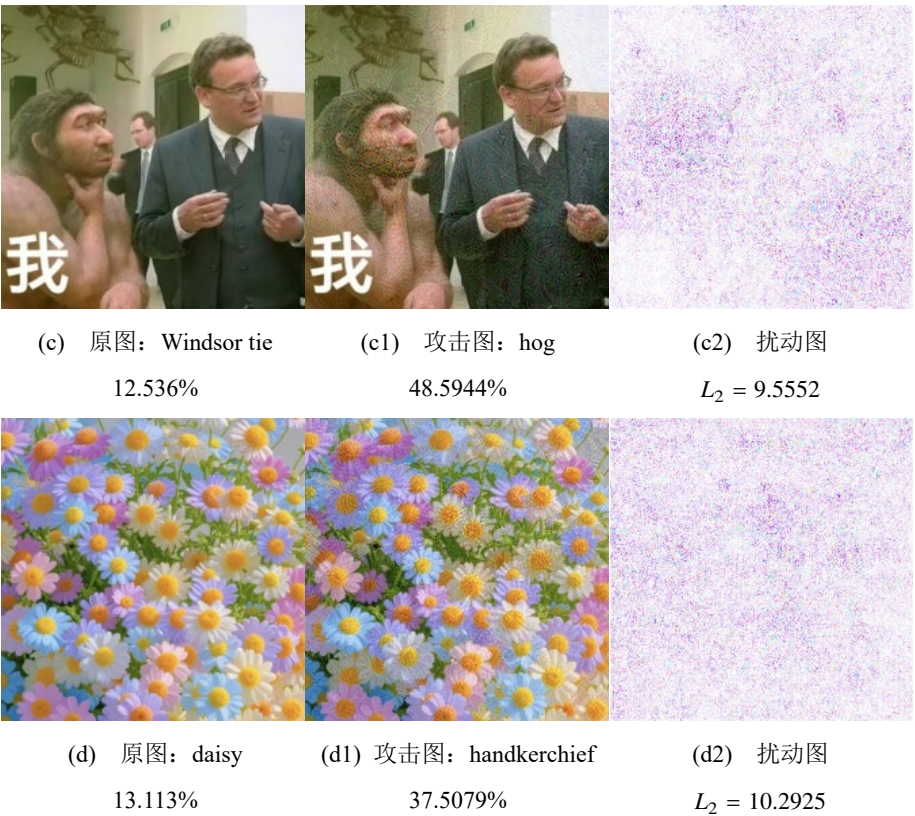


图 4-1 最速下降法实验结果

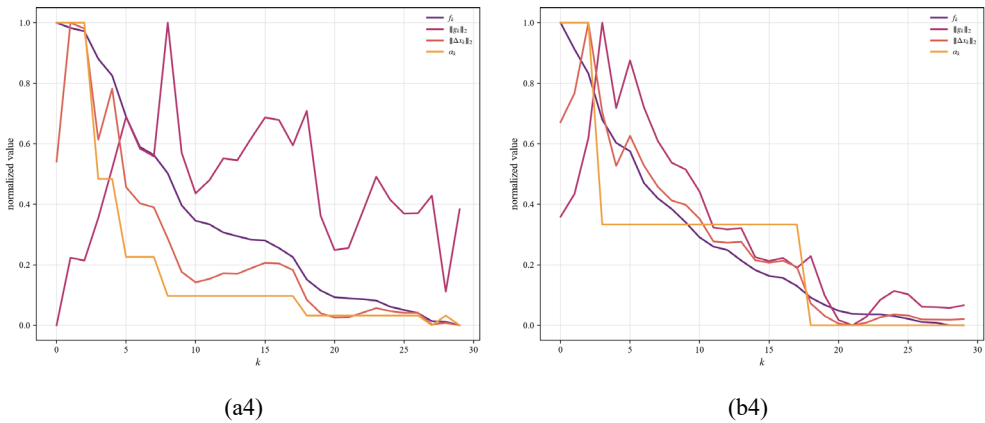
表 4-1 最速下降法实验结果

求解器	成功率	平均耗时/s	最大耗时/s	平均 $\Phi'(\delta^*)/\%$	平均 L_2
manual	4/4	0.238	0.244	39.138	9.728
efficient	4/4	0.499	0.511	82.402	5.843

可见，针对 4 张图片的攻击全部成功。但因为加权法将 $g(\delta) = \epsilon^2 - \|\delta\|_2^2 \geq 0$ 视为软约束，所以扰动强度 L_2 不严格小于 4.0000，肉眼可以发现图 4-1 b 组与 c 组的攻击图与原图的区别。

与最速下降法对比的开源优化器是 Adam 优化器。Adam 优化器同样只考虑一阶信息，但最优步长的选取方法不同，会考虑动量信息。可以看到，Adam 优化器攻击所需的扰动强度 L_2 明显小于我们手搓的结果。

下面观察迭代过程：



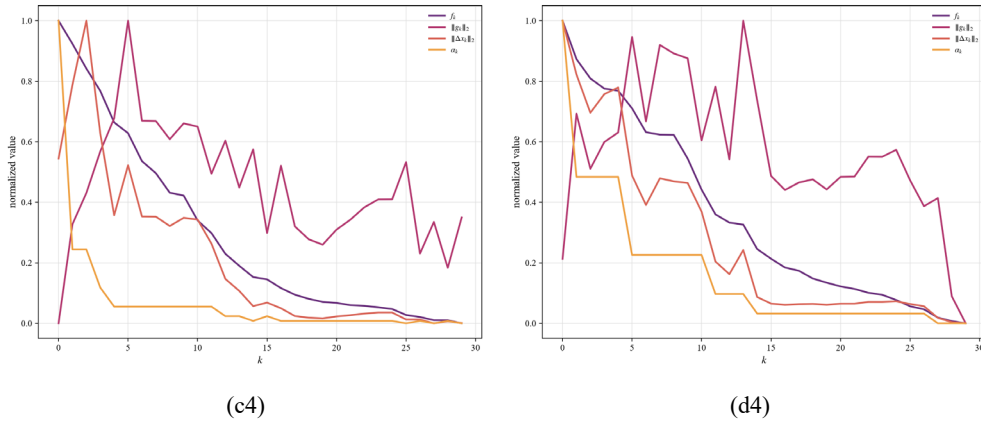


图 4-2 最速下降法迭代过程

可以发现，因为加权法将 $g(\delta) = \varepsilon^2 - \|\delta\|_2^2 \geq 0$ 视为软约束，所以扰动强度 L_2 虽然随迭代总体上呈减小趋势，但也有局部抖动。对于图 4-2 a4 与 c4，可以直观地看出最终解不是 L_2 的全局最优解。

4.3 牛顿法

4.3.1 方法设计

对于加权法问题

$$\min Q(\delta) = \Phi(\delta) + P\|\delta\|_2^2. \quad (4-1)$$

将其近似为二次问题：

$$Q(\delta) \approx Q(\delta^{(k)}) + [\nabla Q(\delta^{(k)})]^\top (\delta - \delta^{(k)}) + \frac{1}{2} (\delta - \delta^{(k)})^\top [\nabla^2 Q(\delta^{(k)})] (\delta - \delta^{(k)}), \quad (4-7)$$

等式两边对 δ 求导，则有：

$$\nabla Q(\delta) \approx \nabla Q(\delta^{(k)}) + [\nabla^2 Q(\delta^{(k)})] (\delta - \delta^{(k)}). \quad (4-8)$$

代入驻点条件，则有：

$$\nabla Q(\delta) \approx \nabla Q(\delta^{(k)}) + [\nabla^2 Q(\delta^{(k)})] (\delta - \delta^{(k)}) = 0, \quad (4-9)$$

整理得迭代公式为：

$$\delta^{(k+1)} = \delta^{(k)} - [\nabla^2 Q(\delta^{(k)})]^{-1} \nabla Q(\delta^{(k)}). \quad (4-10)$$

$$p_{Nt}^{(k)} = -[\nabla^2 Q(\delta^{(k)})]^{-1} \nabla Q(\delta^{(k)}). \quad (4-11)$$

牛顿法在极值点附近收敛速率快。但计算量大，要求二阶导数和 Hessian 矩阵逆，而且因为不知道 $\Phi(\delta)$ 是不是凸函数，所以远离极小点时不一定是下降方向，需进一步修正。这里我们采用 Levenberg-Marquardt 修正：因为若对称阵 $M > 0$ ，则

$$Q(\delta^{(k)} + \lambda_k p^{(k)}) = Q(\delta^{(k)}) - \lambda_k [\nabla Q(\delta^{(k)})]^\top M \nabla Q(\delta^{(k)}) + O(\lambda_k) < Q(\delta^{(k)}), \quad (4-12)$$

所以若对称阵 $M > 0$ ，就能保证 $p^{(k)} = -M \nabla Q(\delta^{(k)})$ 是下降方向。取 L-M 修正方向为：

$$p^{(k)} = -[\nabla^2 Q(\delta^{(k)}) + \mu_k I]^{-1} \nabla Q(\delta^{(k)}) \quad (4-13)$$

$$\mu_k > |\lambda_{\min}|$$

其中 $\lambda_{\min}$ 为 $\nabla^2 Q(\delta^{(k)})$ 的最小负特征值。当 $\mu \rightarrow 0$ ，即为牛顿法；当 $\mu \rightarrow \infty$ ，即为最速下降法。

4.3.2 实验结果



图 4-3 牛顿法实验结果

表 4-2 牛顿法实验结果

求解器	成功率	平均耗时/s	最大耗时/s	平均 $\Phi'(\delta^*)/\%$	平均 L_2
manual	4/4	6.901	10.782	37.421	9.732
efficient	4/4	0.491	0.508	82.402	5.843

可见，实验所得的平均 $\Phi'(\delta^*)$ 、平均 L_2 数据与最速下降法较为接近，可能是因为这 4 张原图处，目标函数的 $\Phi(\delta^*)$ 的二阶导数较小，所以迭代方向主要由一阶信息决定。但平均耗时约为最速下降法的 100 倍，攻击效率较低。

与最速下降法对比的开源优化器是 L-BFGS (Limited-memory Broyden–Fletcher–Goldfarb–Shanno) 优化器，这利用一阶信息近似二阶信息的高效拟牛顿优化器，只保存最近 m 次一阶信息，不需要存储完整 Hessian，因此平均耗时远小于我们的手搓牛顿法。

下面观察迭代过程：

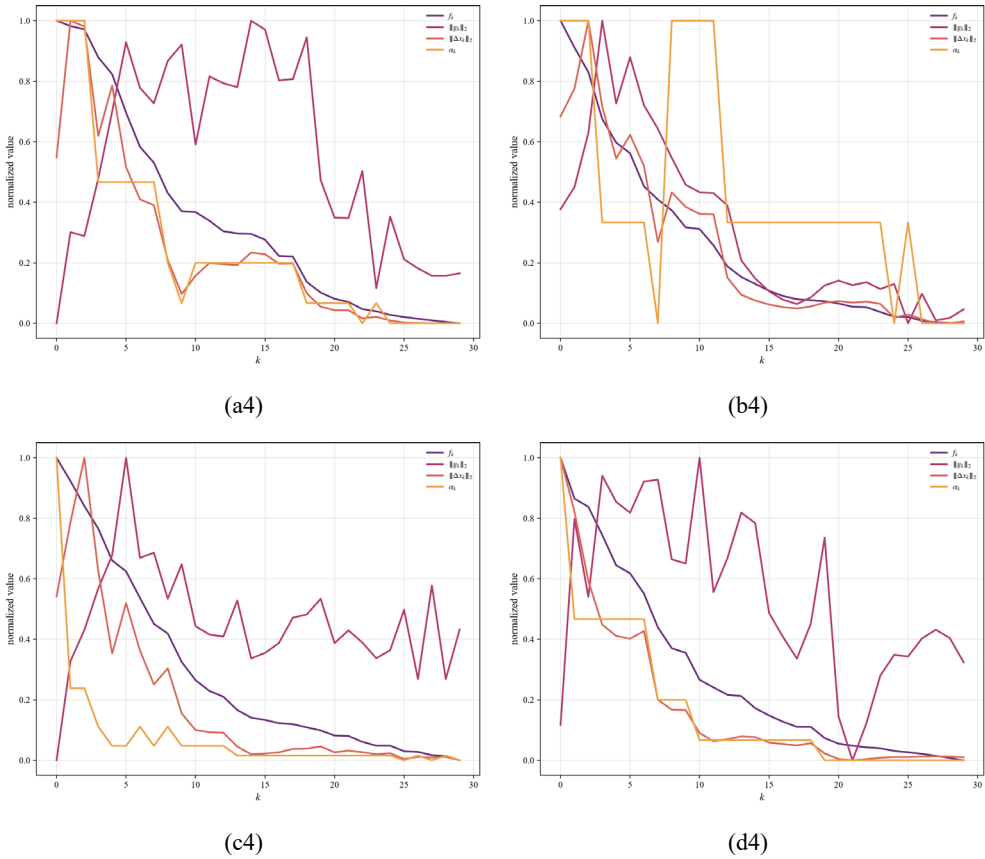


图 4-4 牛顿法迭代过程

同样，扰动强度 L_2 虽然随迭代总体上呈减小趋势，但也有明显的局部抖动。



图 4-5 Levenberg-Marquardt 修正的牛顿法实验结果

表 4-3 Levenberg-Marquardt 修正的牛顿法实验结果

求解器	成功率	平均耗时/s	最大耗时/s	平均 $\Phi'(\delta^*)/\%$	平均 L_2
manual	4/4	6.906	10.502	40.670	9.379
efficient	4/4	0.502	0.515	82.402	5.843

可见，实验所得的平均 $\Phi'(\delta^*)$ 、平均 L_2 数据与最速下降法、牛顿法较为接近，说明无需修正。其实做完牛顿法后，发现所有图片攻击成功，就可以认为所得极值点为极小值点，也就不再做 L-m 修正了。

下面观察迭代过程：

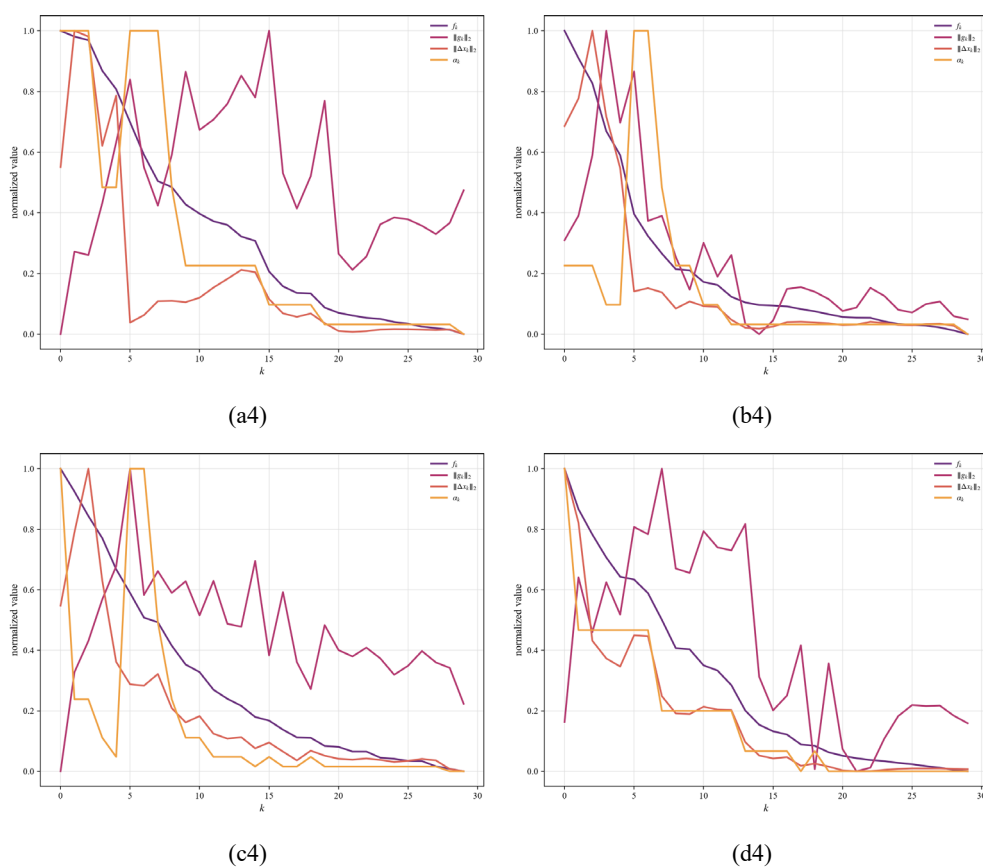


图 4-6 Levenberg-Marquardt 修正的牛顿法迭代过程

可见，相较于牛顿法，扰动强度 L_2 的局部抖动略有改善，这是因为 Levenberg-Marquardt 修正能将迭代方向修正到下降方向。

4.4 拟牛顿法

4.4.1 方法设计

拟牛顿法通过数值法求 Hessian 矩阵的逆。

我们在 4.3.1 节已经证明：

$$\nabla Q(\delta) \approx \nabla Q(\delta^{(k+1)}) + [\nabla^2 Q(\delta^{(k+1)})](\delta - \delta^{(k+1)}) = 0, \quad (4-11)$$

则有：

$$\nabla Q(\delta^{(k)}) \approx \nabla Q(\delta^{(k+1)}) + [\nabla^2 Q(\delta^{(k+1)})](\delta^{(k)} - \delta^{(k+1)}) = 0, \quad (4-14)$$

整理得：

$$\delta^{(k+1)} - \delta^{(k)} \approx [\nabla^2 Q(\delta^{(k+1)})]^{-1} [\nabla Q(\delta^{(k+1)}) - \nabla Q(\delta^{(k)})]. \quad (4-15)$$

设 $\Delta_k \triangleq \delta^{(k+1)} - \delta^{(k)}$, $H_k \approx [\nabla^2 Q(\delta^{(k)})]^{-1}$, $\gamma_k \triangleq \nabla Q(\delta^{(k+1)}) - \nabla Q(\delta^{(k)})$, 则

$$\Delta_k = H_{k+1} \gamma_k, \quad (4-16)$$

称为拟牛顿条件。

下面采用 DFP 变尺度法。迭代方向：

$$p^{(k)} = -H_k \nabla Q(x^{(k)}), \quad (4-17)$$

对应牛顿方向：

$$p^{(k)} = -[\nabla^2 Q(x^{(k)})]^{-1} \nabla Q(x^{(k)}), \quad (4-18)$$

尺度矩阵：

$$H_{k+1} = H_k + \Delta H_k, \quad (4-19)$$

$$H_1 = I,$$

校正矩阵：

$$\Delta H_k = \frac{\delta_k \delta_k^\top}{\delta_k^\top \gamma_k} - \frac{H_k \gamma_k \gamma_k^\top H_k}{\gamma_k^\top H_k \gamma_k}, \quad (4-20)$$

当 H_k 满足拟牛顿条件，为 Hessian 矩阵的逆。

4.4.2 实验结果



(a) 原图：sports car

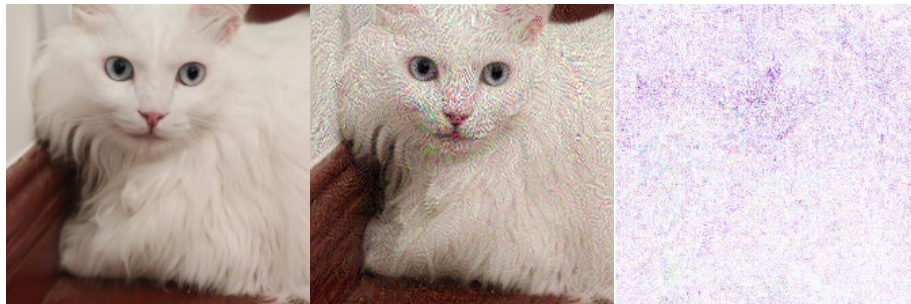
14.517%

(a1) 攻击图：grille

25.3732%

(a2) 扰动图

$L_2 = 7.7795$



(b) 原图：Persian cat

9.729%

(b1) 攻击图：paper towel

51.5791%

(b2) 扰动图

$L_2 = 18.5043$

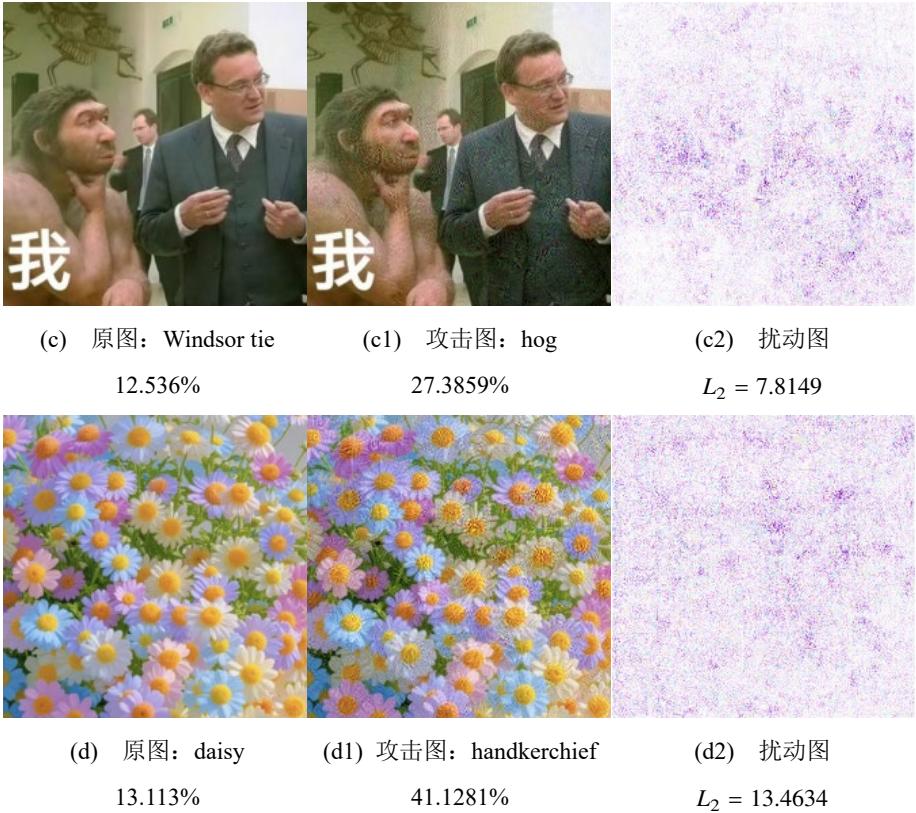


图 4-7 拟牛顿法实验结果

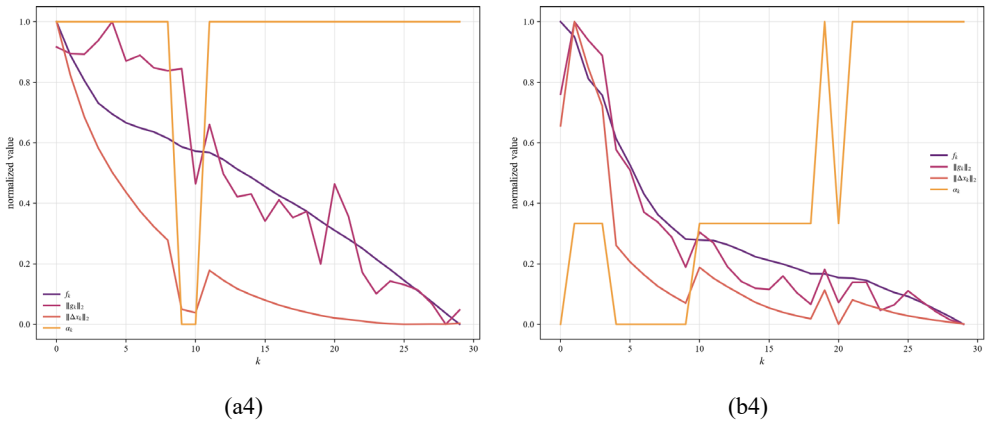
表 4-4 拟牛顿法实验结果

求解器	成功率	平均耗时/s	最大耗时/s	平均 $\Phi'(\delta^*)/\%$	平均 L_2
manual	4/4	0.232	0.244	36.367	11.891
efficient	4/4	0.500	0.517	82.402	5.843

可见，实验所得的平均 $\Phi'(\delta^*)$ 、平均 L_2 数据与牛顿法较为接近，但平均耗时大大减小，甚至小于 L-BFGS 优化器。这说明数值法求 Hessian 矩阵的逆的复杂度远低于直接求逆。

然而，拟牛顿法早于 L-BFGS 优化器停止， L_2 略大于 L-BFGS 优化器，可能是因为拟牛顿法在某个梯度平缓的点（如鞍点）停止，没有找到全局最优解。

下面观察迭代过程：



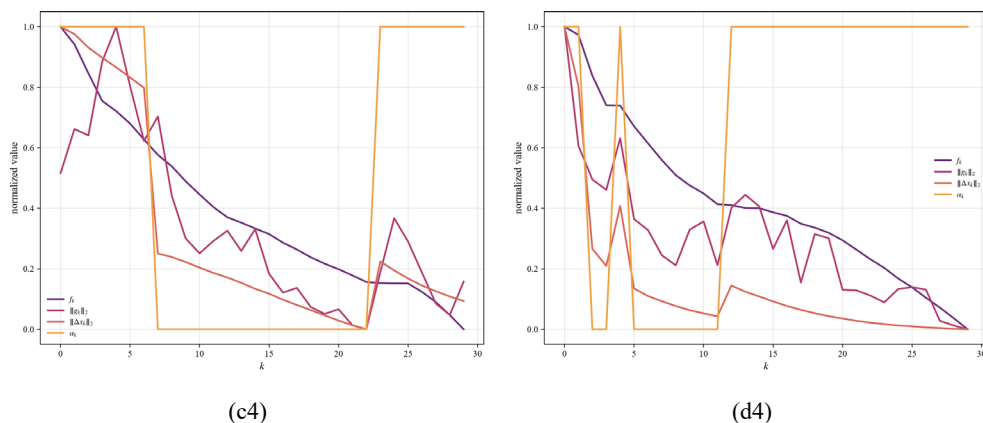


图 4-8 拟牛顿法迭代过程

可见，相较于牛顿法，扰动强度 L_2 的局部抖动有很大改善，而且下降更快。相应地，每步迭代的步长 Δ_k 明显增大。步长过大也可能造成迭代后期在最优解周围震荡，无法收敛。

对比 L-BFGS 优化器的迭代过程：

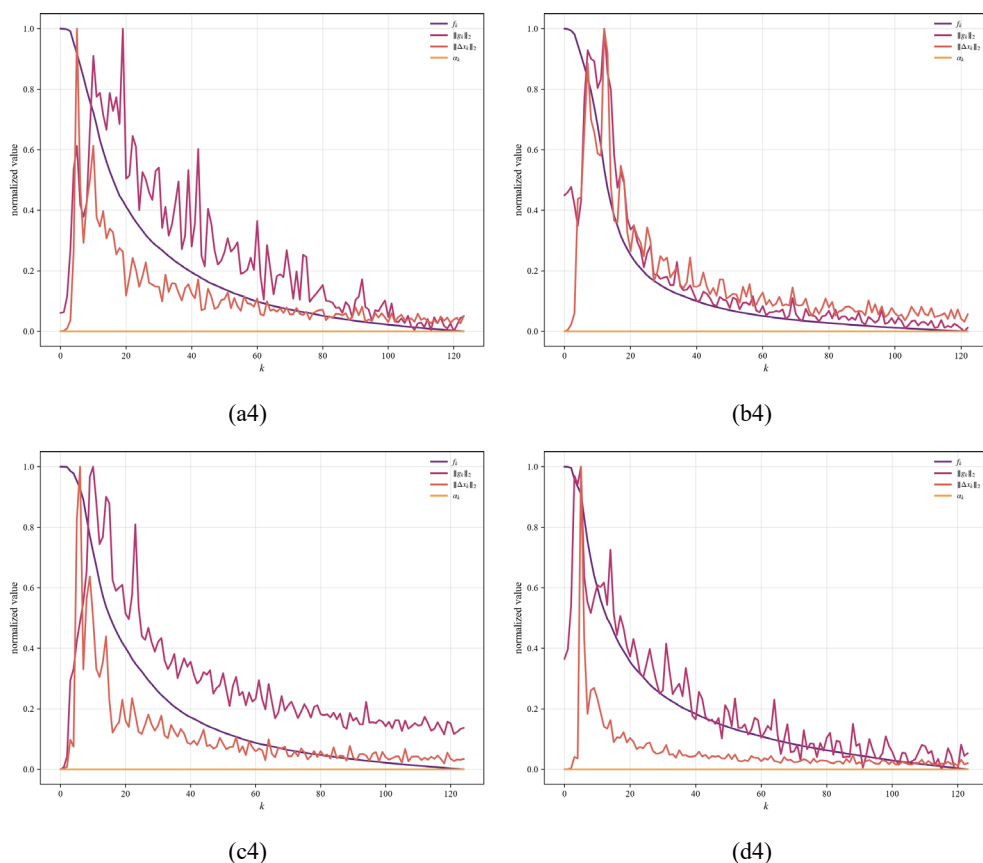


图 4-9 L-BFGS 优化器迭代过程

可以发现，虽然 L-BFGS 优化器的扰动强度 L_2 也存在局部抖动，但是每步迭代的步长 Δ_k 远小于我们的手搓方法，使得下降更平稳。

4.5 共轭梯度法

4.5.1 方法设计

加权法问题

$$\min Q(\delta) = \Phi(\delta) + P\|\delta\|_2^2 \quad (4-1)$$

虽然不是严格凸二次优化问题，但不妨尝试共轭梯度法。

为保证 $p^{(1)}$ 为 $\delta^{(2)}$ 所在等值面的切线方向，取

$$\begin{aligned} p^{(1)} &= -\nabla Q(\delta^{(1)}), \\ \delta^{(2)} &= \delta^{(1)} + \lambda_1^* p^{(1)}. \end{aligned} \quad (4-21)$$

因为 δ^* 未知，所以取共轭方向 $p^{(2)}$ 为：

$$p^{(2)} = -\nabla Q(\delta^{(2)}) + \beta_1 p^{(1)}. \quad (4-22)$$

注意到

$$\begin{aligned} \nabla Q(\delta^{(2)})^\top p^{(2)} &= -\nabla Q(\delta^{(2)})^\top \nabla Q(\delta^{(2)}) + \beta_1 \nabla Q(\delta^{(2)})^\top p^{(1)} = -\|\nabla Q(\delta^{(2)})\|^2 \\ &\leq 0, \end{aligned} \quad (4-23)$$

所以共轭方向 $p^{(2)}$ 一定是下降方向。此时

$$p^{(1)\top} \nabla^2 Q(\delta^{(2)}) p^{(2)} = 0, \quad (4-24)$$

解得

$$\beta_k = \frac{p^{(k)\top} \nabla^2 Q(\delta^{(k+1)}) \nabla Q(\delta^{(k+1)})}{p^{(k)\top} \nabla^2 Q(\delta^{(k+1)}) p^{(k)}}. \quad (4-25)$$

为了避免计算 Hessian 矩阵，可换用以下等价公式：

Crowder-Wolfe 公式：

$$\beta_k = \frac{p^{(k)\top} \nabla^2 Q(\delta^{(k+1)}) \nabla Q(\delta^{(k+1)})}{p^{(k)\top} \nabla^2 Q(\delta^{(k+1)}) p^{(k)}}. \quad (4-26)$$

Fletcher-Reeves 公式：

$$\beta_k = \frac{\nabla Q(\delta^{(k+1)})^\top \nabla Q(\delta^{(k+1)})}{\nabla Q(\delta^{(k)})^\top \nabla Q(\delta^{(k)})}. \quad (4-27)$$

Polak-Ribiere 公式：

$$\beta_k = \frac{\nabla Q(\delta^{(k+1)})^\top [\nabla Q(\delta^{(k+1)}) - \nabla Q(\delta^{(k)})]}{\nabla Q(\delta^{(k)})^\top \nabla Q(\delta^{(k)})}. \quad (4-28)$$

为了保证一般函数下降算法的收敛性，对 β_k 进行自动重置：

$$\beta_k \leftarrow \max\{\beta_k, 0\}. \quad (4-29)$$

共轭梯度法无需计算 Hessian 矩阵，计算量与存储量小，适合本题大规模问题。收敛速度介于最速下降法与牛顿法之间。

4.5.2 实验结果



图 4-10 共轭梯度法, Fletcher-Reeves 公式的实验结果

表 4-5 共轭梯度法，Fletcher-Reeves 公式的实验结果

求解器	成功率	平均耗时/s	最大耗时/s	平均 $\Phi'(\delta^*)/\%$	平均 L_2
manual	4/4	0.271	0.301	37.317	12.986
efficient	4/4	0.494	0.510	82.402	5.843

实验结果令人遗憾：虽然平均耗时远小于牛顿法，攻击效率较高，但平均 $\Phi'(\delta^*)$ 略小于最速下降法，扰动强度 L_2 甚至明显大于最速下降法。尝试观察迭代过程，分析原因：

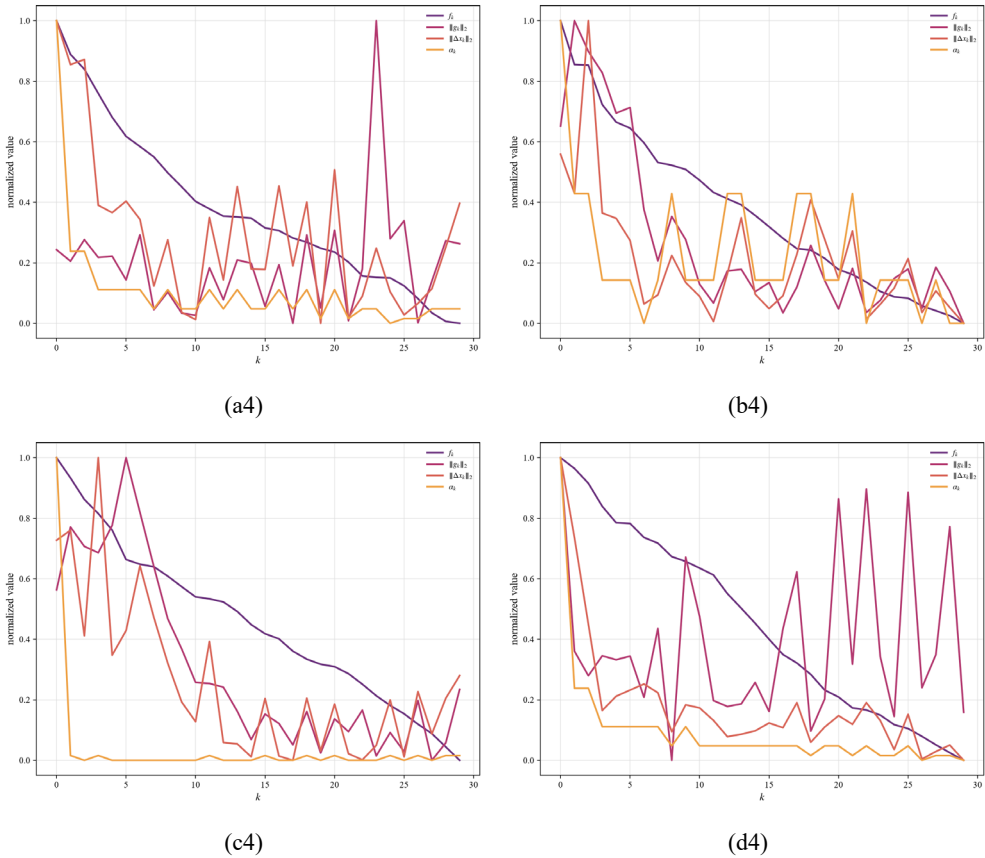


图 4-11 共轭梯度法，Fletcher-Reeves 公式的迭代过程

可以发现，图 4-11 a4 与 d4 的扰动强度 L_2 甚至不呈现下降趋势，几乎没有在迭代过程中优化。我们暂时想到一种猜想：a4 与 d4 在迭代开始时处于一个 L_2 变化平缓的区域，又可见共轭梯度法的最优步长明显小于最速下降法与牛顿法，所以无法冲出该平缓区域，导致无法明显下降。

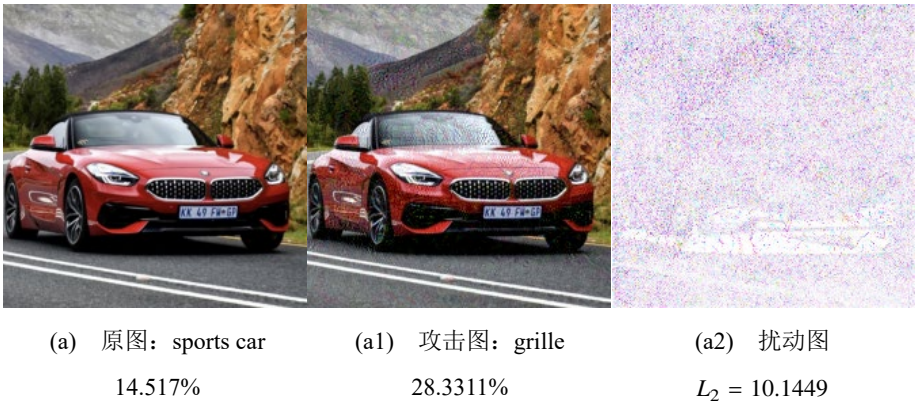




图 4-12 共轭梯度法，Polak-Ribiere 公式的实验结果

表 4-6 共轭梯度法，Polak-Ribiere 公式的实验结果

求解器	成功率	平均耗时/s	最大耗时/s	平均 $\Phi'(\delta^*)/\%$	平均 L_2
manual	4/4	0.280	0.326	39.718	13.623
efficient	4/4	0.497	0.512	82.402	5.843

换用 Polak-Ribiere 等价公式并不能解决 L_2 不被优化的问题，影响共轭梯度法性能的依旧是初始值与步长两个因素，而步长又与初始值有关。因此，影响共轭梯度法性能的主要是初始值（即原图）。我们也许能说：共轭梯度法的迁移性不强，只适合处理严格凸二次优化问题。

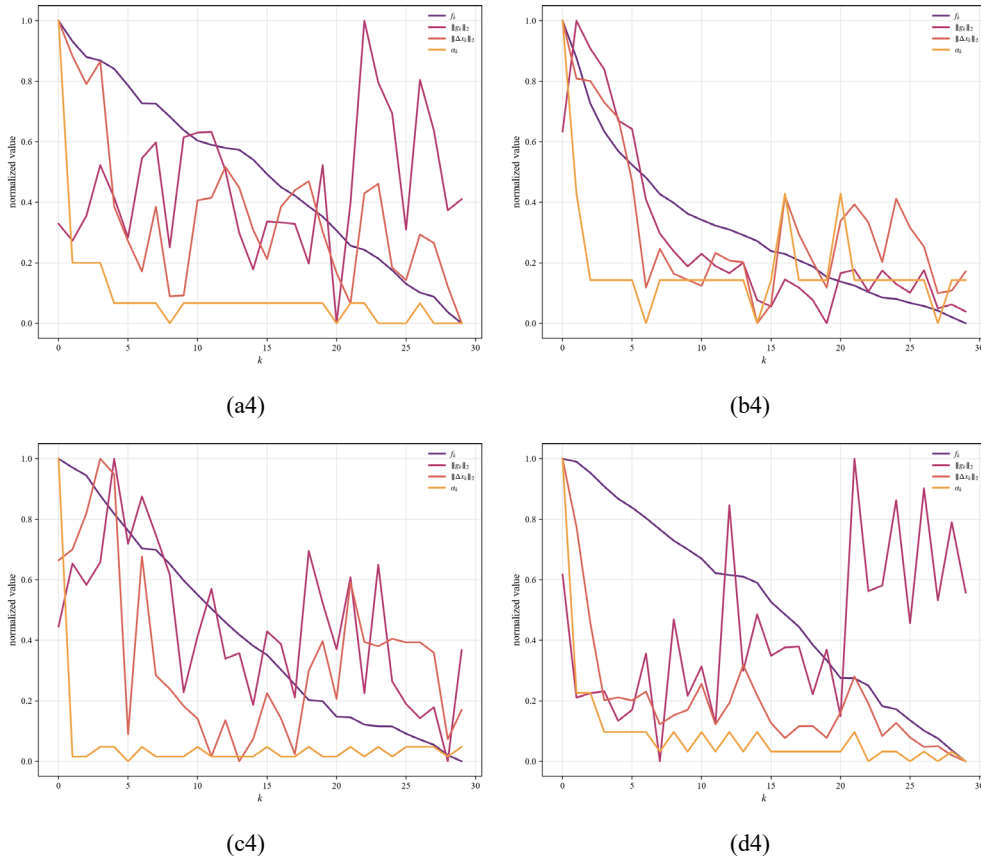


图 4-13 共轭梯度法, Polak-Ribiere 公式迭代过程

4.6 外点法模型

4.6.1 方法设计

对于外点法模型

$$\min P(\delta, M) = \Phi(\delta) + M\{\min[0, g(\delta)]\}^2. \quad (4-2)$$

$M > 0$ 为罚因子, 当 M 趋向无穷时, δ^* 为问题 2-8 的最优解。

令第 k 步罚因子为 M_k , 构造第 k 步罚函数:

$$P(\delta, M_k) = \Phi(\delta) + M_k P_0(\delta). \quad (4-30)$$

因此第 k 步需要求解的无约束极小化问题为 $\min_{\delta \in R^n} P(\delta, M_k)$ 。由于该问题已经是无约束极小化问题, 便可以使用前文的下降迭代法进行求解。考虑到本问题维数很高, 且 Hessian 矩阵规模为 150528×150528 , 若采用牛顿法会产生较大的计算量, 因此本节在每一步无约束极小化中采用一阶下降思想求解。

记 $Q_k(\delta) = P(\delta, M_k)$, 则下降方向取为

$$p^{(r)} = -\nabla Q_k(\delta^{(r)}). \quad (4-31)$$

其中 r 表示在固定罚因子 M_k 下的内层迭代次数。于是迭代格式为

$$\delta^{(r+1)} = \delta^{(r)} + \lambda_r p^{(r)}. \quad (4-32)$$

其中 λ_r 为步长。为保证目标函数下降, 步长可通过一维搜索或预设递减策略确定。

为了进行下降迭代, 需要计算罚函数的梯度。第 k 步罚函数为

$$P(\delta, M_k) = \Phi(\delta) + M_k [\min(0, g(\delta))]^2 = \Phi(\delta) + M_k [\min(0, g(\delta))]^2. \quad (4-33)$$

其中 $\nabla g(\delta) = -2\delta$ 。当 $g(\delta) \geq 0$ 时， $\min(0, g(\delta)) = 0$ ，罚项为 0，因此 $\nabla P(\delta, M_k) = \nabla \Phi(\delta)$ ；当 $g(\delta) < 0$ 时， $\min(0, g(\delta)) = g(\delta)$ ，罚函数为 $P(\delta, M_k) = \Phi(\delta) + M_k [g(\delta)]^2$ ，因此 $\nabla P(\delta, M_k) = \nabla \Phi(\delta) + 2M_k g(\delta) \nabla g(\delta)$ 。代入 $\nabla g(\delta) = -2\delta$ ，得

$$\nabla P(\delta, M_k) = \nabla \Phi(\delta) - 4M_k g(\delta) \delta. \quad (4-34)$$

即

$$\nabla P(\delta, M_k) = \nabla \Phi(\delta) - 4M_k (\varepsilon^2 - \|\delta\|_2^2) \delta. \quad (4-35)$$

综上，罚函数梯度可写为

$$\nabla P(\delta, M_k) = \begin{cases} \nabla \Phi(\delta), & g(\delta) \geq 0; \\ \nabla \Phi(\delta) + 2M_k g(\delta) \nabla g(\delta), & g(\delta) < 0. \end{cases} \quad (4-36)$$

即

$$\nabla P(\delta, M_k) = \begin{cases} \nabla \Phi(\delta), & \varepsilon^2 - \|\delta\|_2^2 \geq 0; \\ \nabla \Phi(\delta) - 4M_k (\varepsilon^2 - \|\delta\|_2^2) \delta, & \varepsilon^2 - \|\delta\|_2^2 < 0. \end{cases} \quad (4-37)$$

接下来分析外点法的收敛性。第 k 步罚函数为

$$P(\delta, M_k) = \Phi(\delta) + M_k [\min(0, g(\delta))]^2. \quad (4-30)$$

若第 k 步罚函数的局部极小值为 $\delta^{(k)}$ ，则满足一阶驻点条件：

$$\nabla P(\delta^{(k)}, M_k) = 0. \quad (4-38)$$

当约束被违反时，即 $g(\delta^{(k)}) < 0$ ，有

$$\nabla P(\delta^{(k)}, M_k) = \nabla \Phi(\delta^{(k)}) + 2M_k g(\delta^{(k)}) \nabla g(\delta^{(k)}) = 0. \quad (4-39)$$

将其写为 KKT 条件的形式：

$$\nabla \Phi(\delta^{(k)}) - \mu^{(k)} \nabla g(\delta^{(k)}) = 0. \quad (4-40)$$

其中 $\mu^{(k)} = -2M_k g(\delta^{(k)})$ 。由于此时 $g(\delta^{(k)}) < 0$ ，且 $M_k > 0$ ，所以 $\mu^{(k)} \geq 0$ 。当 $\delta^{(k)}$ 迭代收敛时，有 $g(\delta^*) \rightarrow 0$ ，于是可得

$$\begin{aligned} \nabla \Phi(\delta^*) - \mu^* \nabla g(\delta^*) &= 0; \\ \mu^* &\geq 0; \\ g(\delta^*) &\geq 0; \\ \mu^* g(\delta^*) &= 0. \end{aligned} \quad (4-41)$$

这正是原约束问题的 KKT 条件。

因此，外点法在迭代收敛时，能够得到满足 KKT 条件的点。若每一步所求得的 $\delta^{(k)}$ 都是 $\min P(\delta, M_k)$ 的全局极小值，则外点法收敛到原问题的全局最优解。

但是，本文中的 $\Phi(\delta)$ 是由 MobileNetV2 的复杂网络结构得到的非线性函数，其凹凸性无法确定，因此实际计算中不能保证每一步无约束问题都达到全局极小值。所以，本节外点法所得结果通常只能认为是满足约束意义下的近似极小解。

4.6.2 实验结果



图 4-14 外点法实验结果

表 4-7 外点法实验结果

求解器	成功率	平均耗时/s	最大耗时/s	平均 $\Phi'(\delta^*)/\%$	平均 L_2
manual	4/4	0.234	0.250	9.681	6.655
efficient	4/4	0.831	1.362	5.004	0.255

想必读者已经发现：外点法的扰动图与其他任何一种方法都不一样！外点法的扰动图颜色颗粒更大，而其他方法的扰动图颜色颗粒更细小。这是因为其他方法的扰动图是由一阶与二阶信息自然生成的，而外点法存在截断的过程：

$$\min P(\delta, M) = \Phi(\delta) + M\{\min[0, g(\delta)]\}^2. \quad (4-42)$$

自然的扰动图颜色是连续变化的，因此颜色颗粒几乎就是单个像素；被截断后，紧挨着的多个像素颜色更可能被截断为一种相同的颜色，这几个紧挨着的像素就组成了一个粗大的颜色颗粒。

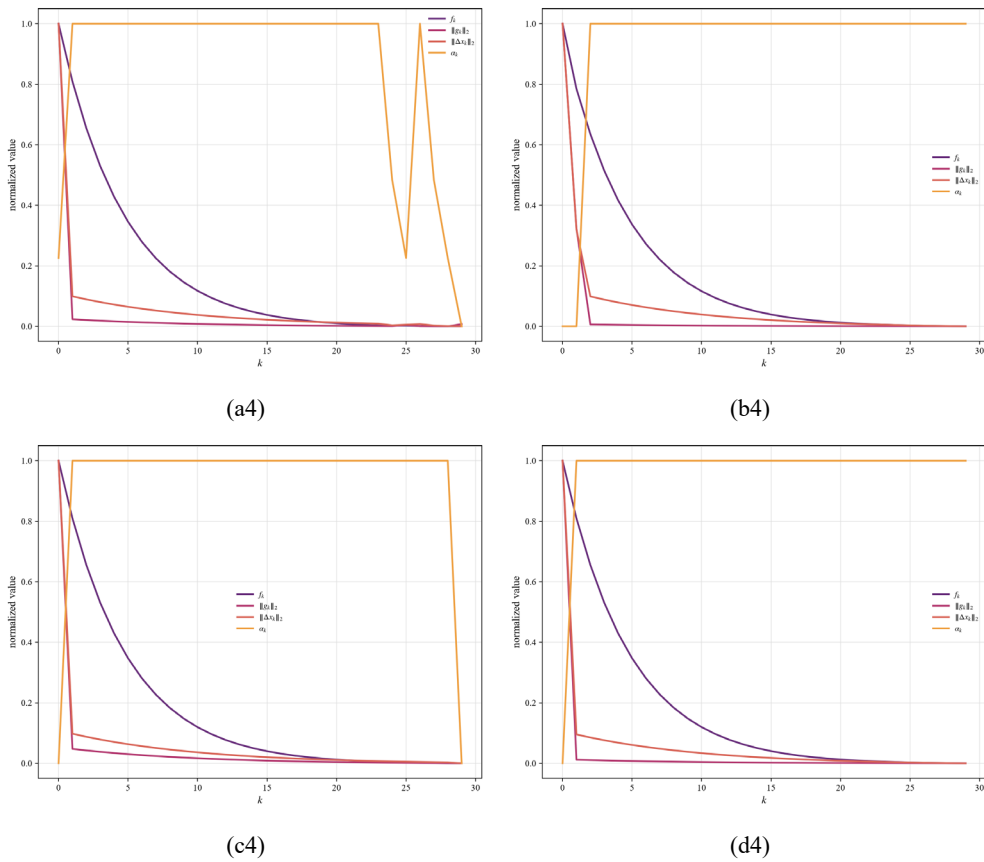


图 4-15 外点法迭代过程

4.7 投影法模型

4.7.1 方法设计

投影法模型只需改变加权法模型的迭代终止条件即可，不再赘述。

4.7.2 实验结果



图 4-16 投影法实验结果

表 4-8 投影法实验结果

求解器	成功率	平均耗时/s	最大耗时/s	平均 $\Phi'(\delta^*)/\%$	平均 L_2
manual	4/4	0.229	0.234	31.465	2.629
efficient	4/4	0.218	0.222	31.465	2.629

投影法是迭代法中对扰动强度 L_2 限制最严格的方法，肉眼极难分辨攻击图与原图的区别。

下面展示迭代过程：

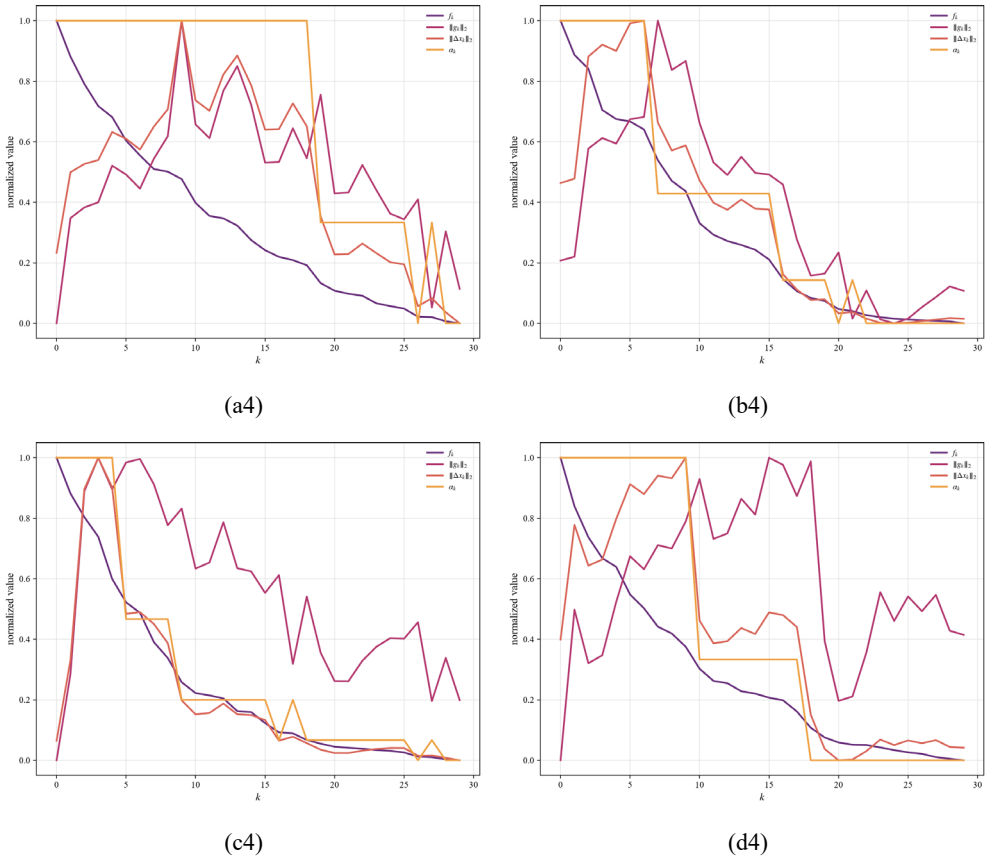


图 4-17 投影法迭代过程

虽然 L_2 同样存在抖动，但只要超出 ε^2 就会退回。

5 化为对偶问题

在前几章中，我们针对的都是问题 2-8：

$$\begin{aligned} \min_{\delta \in \mathbb{R}^n} \quad & \Phi(\delta) = z_{\text{Are you a balabala?}}(x + \delta) - \log \left[\sum_{j=1}^{1000} e^{z_j(x+\delta)} \right] \\ \text{s.t.} \quad & g(\delta) = \varepsilon^2 - \|\delta\|_2^2 \geq 0. \end{aligned} \quad (2-8)$$

即

$$\boxed{\text{限制最大的扰动，追求最大的破坏}} \quad (5-1)$$

我们很容易联想到，能否将其化为对偶问题呢？即

$$\boxed{\text{限制最小的破坏，追求最小的扰动}} \quad (5-2)$$

规定原问题的目标函数 $\Phi(\delta)$ 至少降低至阈值 η ，求解扰动强度 $\|\delta\|_2^2$ 的最小值，即：

$$\begin{aligned} \min_{\delta \in \mathbb{R}^n} \quad & \|\delta\|_2^2 \\ \text{s.t.} \quad & \Phi(\delta) \leq \eta. \end{aligned} \quad (5-3)$$

化为标准型：

$$\begin{aligned} \min_{\delta \in \mathbb{R}^n} \quad & \|\delta\|_2^2 \\ \text{s.t.} \quad & \eta - \Phi(\delta) \geq 0. \end{aligned} \quad (5-4)$$

5.1 方法设计

为避免与线性规划中严格意义上的对偶问题混淆，我们改称式 5-4 为式 2-8 的“对偶形式”或“反向约束形式”。二者并不是线性规划中 $\max z = cx, Ax \leq b, x \geq 0$ 与 $\min w = b^T y, A^T y \geq c^T, y \geq 0$ 之间那种严格的对偶关系，而是从攻击目标和扰动约束的角度，将目标和约束进行了交换。

5.1.1 弱对偶性分析

线性规划弱对偶性为：原问题可行解的目标函数值小于等于对偶问题可行解的目标函数值。在线性规划标准型中，若原问题为

$$\begin{aligned} \max \quad & z = cx \\ \text{s.t.} \quad & Ax \leq b \\ & x \geq 0, \end{aligned} \quad (5-5)$$

对偶问题为

$$\begin{aligned} \min \quad & w = b^T y \\ \text{s.t.} \quad & A^T y \geq c^T \\ & y \geq 0, \end{aligned} \quad (5-6)$$

则设 x^0 、 y^0 分别为原始问题和对偶问题的可行解，有

$$z = cx^0 \leq y^{0T} Ax^0 \leq y^{0T} b = w \quad (5-7)$$

从而得到弱对偶性。但是，对于本文中的两个非线性规划问题：

$$\begin{aligned} \min_{\delta \in \mathbb{R}^n} \quad & \Phi(\delta) = z_{\text{Are you a balabala?}}(x + \delta) - \log \left[\sum_{j=1}^{1000} e^{z_j(x+\delta)} \right] \\ \text{s.t.} \quad & g(\delta) = \varepsilon^2 - \|\delta\|_2^2 \geq 0. \end{aligned} \quad (2-8)$$

$$\begin{aligned} \min_{\delta \in \mathbb{R}^n} \quad & \|\delta\|_2^2 \\ \text{s.t.} \quad & \eta - \Phi(\delta) \geq 0. \end{aligned} \quad (5-4)$$

二者的目标函数分别为 $\Phi(\delta)$ 和 $\|\delta\|_2^2$ ，量纲和含义不同，且两个问题均为极小化问题，因此，不能直接比较二者目标函数值的大小。也就是说，一般不能推出类似 $\Phi(\delta^0) \leq \|\delta^1\|_2^2$ 或 $\|\delta^1\|_2^2 \leq \Phi(\delta^0)$ 这样的结论，不满足线性规划意义下的弱对偶性。

不过，若从 Lagrange 对偶的角度考虑式 5-4，可以构造其 Lagrange 函数。令 $h(\delta) = \eta - \Phi(\delta)$ ，则式 5-4 可写为

$$\begin{aligned} \min_{\delta \in \mathbb{R}^n} \quad & \|\delta\|_2^2 \\ \text{s.t.} \quad & h(\delta) = \eta - \Phi(\delta) \geq 0. \end{aligned} \quad (5-8)$$

定义 Lagrange 函数

$$L(\delta, \lambda) = \|\delta\|_2^2 - \lambda h(\delta) = \|\delta\|_2^2 - \lambda[\eta - \Phi(\delta)], \lambda \geq 0. \quad (5-9)$$

对于任意可行解 δ^0 ，有 $h(\delta^0) \geq 0$ ，因此 $-\lambda h(\delta^0) \leq 0$ 。于是

$$L(\delta^0, \lambda) = \|\delta^0\|_2^2 - \lambda h(\delta^0) \leq \|\delta^0\|_2^2. \quad (5-10)$$

又因为

$$\inf_{\delta \in \mathbb{R}^n} L(\delta, \lambda) \leq L(\delta^0, \lambda), \quad (5-11)$$

所以有

$$\inf_{\delta \in \mathbb{R}^n} L(\delta, \lambda) \leq \|\delta^0\|_2^2. \quad (5-12)$$

令 $\theta(\lambda) = \inf_{\delta \in \mathbb{R}^n} L(\delta, \lambda)$ ，则 $\theta(\lambda) \leq \|\delta^0\|_2^2$ 。因此，对 Lagrange 对偶问题 $\max_{\lambda \geq 0} \theta(\lambda)$ ，有

$$\max_{\lambda \geq 0} \theta(\lambda) \leq \min_{\delta \in \mathbb{R}^n} \|\delta\|_2^2. \quad (5-13)$$

可见：式 5-4 与其 Lagrange 对偶问题满足弱对偶性。但这属于 Lagrange 对偶意义下的弱对偶性，而不是式 2-8 与式 5-4 之间的线性规划弱对偶性。

5.1.2 最优性分析

线性规划最优性定理为：设 x^0 、 y^0 分别是原始问题和对偶问题的可行解，则当 $cx^0 = b^\top y^0$ 时， x^0 、 y^0 均为最优解。其证明依赖于弱对偶性，即 $cx^0 \leq cx^* \leq b^\top y^* \leq b^\top y^0$ ；若 $cx^0 = b^\top y^0$ ，则有 $cx^0 = cx^* = b^\top y^* = b^\top y^0$ ，因此二者均为最优解。

对于式 2-8 与式 5-4，由于二者不满足弱对偶性，因此不能直接使用上述最优性定理。也就是说，即使存在某个 δ^0 同时满足两个问题的约束，也不能仅凭某种目标函数相等来判断其为两个问题的最优解。

但是，对于式 5-4 与其 Lagrange 对偶问题，若存在可行解 δ^0 与乘子 $\lambda^0 \geq 0$ ，满足 $\|\delta^0\|_2^2 = \theta(\lambda^0)$ ，则由弱对偶性知，对任意可行解 δ 和任意 $\lambda \geq 0$ ，有 $\theta(\lambda) \leq \|\delta\|_2^2$ 。特别地， $\theta(\lambda^0) \leq \|\delta\|_2^2$ 。又因为 $\|\delta^0\|_2^2 = \theta(\lambda^0)$ ，所以 $\|\delta^0\|_2^2 \leq \|\delta\|_2^2$ ，故 δ^0 为式 5-4 的最优解， λ^0 为其 Lagrange 对偶问题的最优解。

5.1.3 强对偶性分析

线性规划强对偶性为：若原问题和对偶问题均有可行解，则必存在最优解 x^* 、 y^* ，且有 $cx^* = b^\top y^*$ 。

在线性规划中，强对偶性成立的重要原因是可行域为凸集，目标函数为线性函数，因此可通过单纯形法中的最优基构造对偶可行解。

而本文式 5-4 为非线性规划问题：

$$\begin{aligned} \min_{\delta \in \mathbb{R}^n} \quad & \|\delta\|_2^2 \\ \text{s.t.} \quad & \eta - \Phi(\delta) \geq 0. \end{aligned} \quad (5-4)$$

其中目标函数 $\|\delta\|_2^2$ 是凸函数，但约束函数 $\eta - \Phi(\delta)$ 是否为凹函数，取决于 $\Phi(\delta)$ 是否为凸函数。

若 $\Phi(\delta)$ 为凸函数，则 $-\Phi(\delta)$ 为凹函数，因此 $\eta - \Phi(\delta)$ 也是凹函数。此时约束 $\eta - \Phi(\delta) \geq 0$ 对应的是凸优化中的可行域形式，式 5-4 可视为凸规划问题。若进一步存在严格可行解 δ ，满足 $\eta - \Phi(\delta) > 0$ ，则在通常的凸规划条件下，式 5-4 与其 Lagrange 对偶问题之间可认为满足强对偶性，即存在 δ^* 、 λ^* ，使得 $\|\delta^*\|_2^2 = \theta(\lambda^*)$ 。

但是，本文前文已经说明， $\Phi(\delta)$ 是由卷积、批归一化、ReLU6 激活、深度可分离卷积、倒残差结构、全局平均池化和线性分类层等复杂结构得到的非线性函数，其凹凸性无法确定。**因此，不能保证 $\eta - \Phi(\delta)$ 为凹函数。所以，在本文实际问题中，强对偶性不能保证成立。换言之，式 5-4 与其 Lagrange 对偶问题可能存在对偶间隙，即可能有 $\max_{\lambda \geq 0} \theta(\lambda) < \min_{\delta \in \mathbb{R}^n} \|\delta\|_2^2$ 。因此，本节方法求得的解仍只能保证为局部意义下的可行解或近似最优解，不能保证为全局最优解。**

5.1.4 互补松弛性分析

线性规划互补松弛性为：可行解 x^0 、 y^0 分别是原始问题和对偶问题最优解的充要条件是 $(Ax^0 - b)^\top y^0 = 0$ 和 $x^{0\top} (A^\top y^0 - c^\top) = 0$ ，或 $x_s^{0\top} y^0 = 0$ 和 $x^{0\top} y_s^0 = 0$ 。对于式 5-4，只有一个不等式约束 $h(\delta) = \eta - \Phi(\delta) \geq 0$ ，其 Lagrange 乘子为 $\lambda \geq 0$ 。因此，互补松弛条件化为

$$\lambda^* h(\delta^*) = \lambda^* [\eta - \Phi(\delta^*)] = 0. \quad (5-14)$$

若 $\lambda^* > 0$ ，则必须有 $\eta - \Phi(\delta^*) = 0$ ，即 $\Phi(\delta^*) = \eta$ ，此时攻击效果约束恰好起作用；若 $\eta - \Phi(\delta^*) > 0$ ，则必须有 $\lambda^* = 0$ ，此时攻击效果约束不起作用。但对于本问题，目标是求满足攻击效果的最小扰动。若存在 $\eta - \Phi(\delta^*) > 0$ ，则说明攻击效果超过了阈值，即扰动可能仍有缩小空间。因此，在通常情况下，**最优解应位于边界上，即 $\Phi(\delta^*) = \eta$ 。这也与前文 Lagrange 函数法中的思想一致：最小的扰动通常发生在攻击效果约束刚好取等号的位置。**

5.1.5 方法设计

结合前文实验结果，KKT 条件法需要计算 Hessian 矩阵 $\nabla_x^2 \Phi(0)$ ，而本问题维数为 $n = 150528$ ，因此 Hessian 矩阵规模为 150528×150528 ，计算和存储代价很大。相比之下，3.2 节中的 Lagrange 函数法只需要一阶梯度，平均耗时约为 0.1 秒，且攻击成功率为 4/4。因此，本节采用与 3.2 节一致的思想，对式 5-4 进行求解。

将式 5-4 化为等式约束问题：

$$\begin{aligned} \min_{\delta \in \mathbb{R}^n} \quad & \|\delta\|_2^2 \\ \text{s.t.} \quad & \eta - \Phi(\delta) = 0. \end{aligned} \quad (5-15)$$

定义 Lagrange 函数：

$$L(\delta, \lambda) = \|\delta\|_2^2 + \lambda [\Phi(\delta) - \eta], \quad (5-16)$$

则有

$$\begin{cases} \frac{\partial L(\delta, \lambda)}{\partial \delta} = 2\delta + \lambda \nabla \Phi(\delta); \\ \frac{\partial L(\delta, \lambda)}{\partial \lambda} = \Phi(\delta) - \eta. \end{cases} \quad (5-17)$$

最优解应满足

$$\begin{cases} 2\delta^* + \lambda^* \nabla \Phi(\delta^*) = 0; \\ \Phi(\delta^*) - \eta = 0. \end{cases} \quad (5-18)$$

由于 $\Phi(\delta)$ 是复杂非线性函数，直接求解上述方程组较困难。与 3.2 节相同，对 $\Phi(\delta)$ 在 $\delta = 0$ 处作一阶近似：

$$\Phi(\delta) \approx \Phi(0) + [\nabla_x \Phi(0)]^\top \delta. \quad (5-19)$$

记 $d = \nabla_x \Phi(0)$ ，则有 $\Phi(\delta) \approx \Phi(0) + d^\top \delta$ ，代入约束 $\Phi(\delta) = \eta$ ，得到 $\Phi(0) + d^\top \delta = \eta$ ，即

$$d^\top \delta = \eta - \Phi(0) \quad (5-20)$$

因为攻击的目的是降低目标函数值，通常取 $\eta < \Phi(0)$ ，于是 $\eta - \Phi(0) < 0$ 。此时问题近似化为

$$\begin{aligned} \min_{\delta \in \mathbb{R}^n} \quad & \|\delta\|_2^2 \\ \text{s.t.} \quad & d^\top \delta = \eta - \Phi(0). \end{aligned} \quad (5-21)$$

再次定义 Lagrange 函数：

$$L(\delta, \lambda) = \delta^\top \delta + \lambda [d^\top \delta - \eta + \Phi(0)]. \quad (5-22)$$

由驻点条件，有

$$\frac{\partial L(\delta, \lambda)}{\partial \delta} = 2\delta + \lambda d = 0. \quad (5-23)$$

因此 $\delta = -\frac{\lambda}{2}d$ ，代入约束 $d^\top \delta = \eta - \Phi(0)$ ，得

$$d^\top \left(-\frac{\lambda}{2}d \right) = \eta - \Phi(0), \quad (5-24)$$

即

$$-\frac{\lambda}{2}d^\top d = \eta - \Phi(0). \quad (5-25)$$

解得

$$\lambda = \frac{2[\Phi(0) - \eta]}{d^\top d}, \quad (5-26)$$

于是

$$\delta^* = -\frac{\Phi(0) - \eta}{d^\top d}d, \quad (5-27)$$

即

$$\delta^* = -\frac{\Phi(0) - \eta}{[\nabla_x \Phi(0)]^\top [\nabla_x \Phi(0)]} \nabla_x \Phi(0). \quad (5-28)$$

该式即为本节采用的扰动方向与扰动大小。其含义十分直观： $-\nabla_x \Phi(0)$ 是使目标函数 $\Phi(\delta)$ 下降最快的方向； $\Phi(0) - \eta$ 表示希望目标函数下降的幅度； $[\nabla_x \Phi(0)]^\top [\nabla_x \Phi(0)]$ 用于归一化，使扰动刚好达到一阶近似下的阈值要求。

由此可得近似最小扰动强度为

$$\|\delta^*\|_2^2 = \left\| -\frac{\Phi(0) - \eta}{d^\top d}d \right\|_2^2, \quad (5-29)$$

即

$$\|\delta^*\|_2^2 = \frac{[\Phi(0) - \eta]^2}{d^\top d}, \quad (5-30)$$

代回 $d = \nabla_x \Phi(0)$ ，有

$$\|\delta^*\|_2^2 = \frac{[\Phi(0) - \eta]^2}{[\nabla_x \Phi(0)]^\top [\nabla_x \Phi(0)]}. \quad (5-31)$$

5.2 实验结果



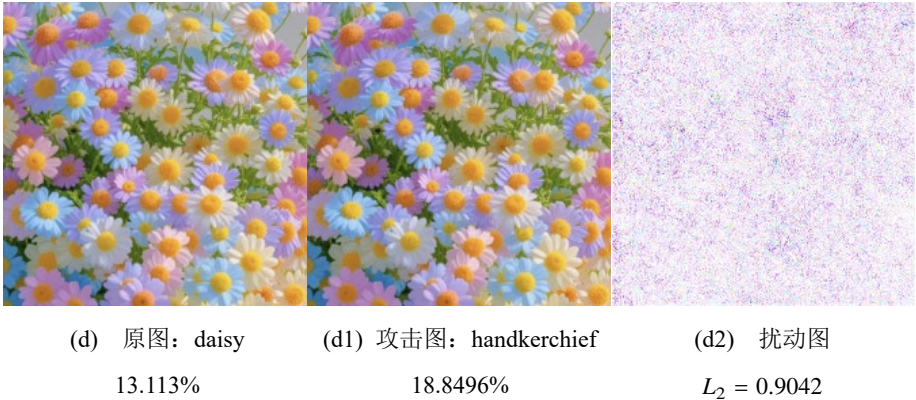


图 5-1 化为对偶问题实验结果

表 5-1 化为对偶问题实验结果

求解器	成功率	平均耗时/s	最大耗时/s	平均 $\Phi'(\delta^*)/\%$	平均 L_2
manual	4/4	0.990	1.017	23.607	1.416
efficient	4/4	1.990	1.999	64.246	2.759

化为对偶问题并求解是我们实验中的最佳方法。不但决策函数能达到很高的阈值，而且所需的扰动强度 L_2 远小于针对原问题的方法，肉眼几乎无法分辨攻击图上的扰动噪声。

这也说明本章所谓的“对偶问题”与原问题之间不满足弱对偶性、最优性、强对偶性，二者的最优解不同。

6 让模型将任何东西识别为厕纸

前文尝试的攻击方法只追求让被攻击模型产生错误，而不关心产生怎样的错误。接下来，我们尝试让被攻击模型产生指定的错误，即把任何东西都识别为某一特定物品。

MobileNetV2 对应 ImageNet-1K 数据集的 1000 类物品，其中第 1000 类 ($j = 999$) 物品即为厕纸 (toilet tissue)，因此，不妨选取厕纸作为指定目标。

如果读者已经忘了视觉分类模型的决策过程，请再看一眼：

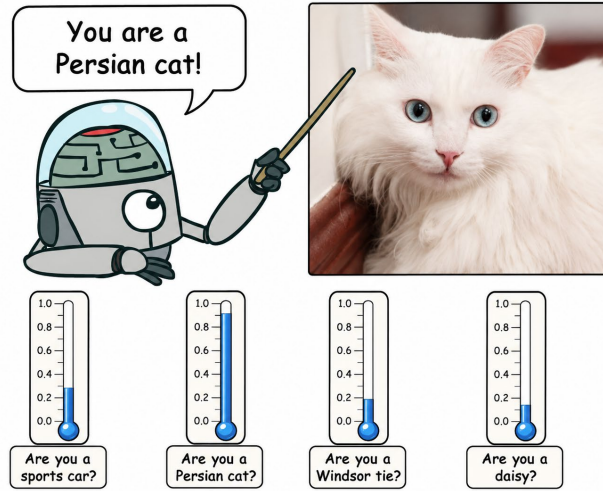


图 2-2 视觉分类模型的决策过程。下方的“温度计”表示各个类别的得分函数。

视觉模型在读入原图 x 之后，会分别求出 1000 个类别对应的得分函数 $z_1(x), z_2(x), \dots, z_{1000}(x)$ ，例如

$$\begin{aligned} z_{817}(x) &= z_{\text{Are you a sports car?}}(x), \\ z_{283}(x) &= z_{\text{Are you a Persian cat?}}(x), \\ z_{701}(x) &= z_{\text{Are you a Windsor tie?}}(x), \\ z_{985}(x) &= z_{\text{Are you a daisy?}}(x), \\ &\dots \end{aligned} \tag{2-2}$$

然后取

$$\widehat{\text{balabala}}(x) = \arg\{\max[z_{\text{Are you a balabala?}}(x)]\}, \tag{2-3}$$

$\widehat{\text{balabala}}$ 即为模型所认为的这张图片应归属的类。

现在我们要让 $\widehat{\text{balabala}} = \text{toilet tissue}$ ，就得让 $z_{\text{Are you a toilet tissue?}}(x + \delta)$ 尽可能大，即

$$\begin{aligned} \min_{\delta \in \mathbb{R}^n} \quad & \|\delta\|_2^2 \\ \text{s.t.} \quad & z_{\text{Are you a toilet tissue?}}(x + \delta) - \log \left[\sum_{j=1}^{1000} e^{z_j(x + \delta)} \right] - \eta' \geq 0. \end{aligned} \tag{6-1}$$

求解方法与第 5 章完全相同，不再赘述。

6.1 实验结果



图 6-1 让模型将任何东西识别为厕纸实验结果

表 6-1 让模型将任何东西识别为厕纸实验结果

求解器	成功率	平均耗时/s	最大耗时/s	平均 $\Phi'(\delta^*)/\%$	平均 L_2
manual	4/4	1.120	1.155	3.142	1.174
efficient	4/4	2.128	2.180	7.670	1.407

可见，模型将 4 张攻击图片全部识别为厕纸，攻击成功。因为本章同样利用了化为对偶问题的思想，所以攻击成功所需的扰动二范数非常小。可以发现，图 6-1 b2 的扰动图几乎是纯白色的，即几乎不需要扰动，模型就很容易将图 6-1 b 识别为厕纸。

7 面积最小的“张天羽”

前文尝试的攻击方法都针对非线性规划模型，追求使图像扰动无法被敌人肉眼察觉。为了联系图论的知识，本章考虑另一种情境：为了表达对敌人的嘲讽，在图片上绘制纯黑色的文字或图案，例如“张天羽”三字，在攻击成功的前提下，使“张天羽”的面积尽量小。

该问题同时包含攻击约束、字形约束和连通约束三类条件。首先，置黑后的图像应当能够使被攻击模型产生错误输出；其次，被置黑像素应位于给定文字模板内，保留各笔画的基本结构；最后，同一笔画中被保留的像素应当在图像网格上相互连通。因此，我们将该问题化为一个带图结构约束的0-1非线性规划问题，并利用网络流模型刻画笔画连通性。

7.1 建立模型

7.1.1 图像网格图

设输入图像的高度和宽度分别为 H 和 W ，像素位置总数为 $m = HW$ 。将每个像素位置视为一个顶点，记顶点集合为 $V = \{v_1, v_2, \dots, v_m\}$ 。若两个像素在空间上上下相邻或左右相邻，则在对应顶点之间连接一条无向边。由此得到图像四邻域网格图

$$G = (V, E), \quad (7-1)$$

其中边集合为

$$E = \{\{v_i, v_j\} : v_i \text{与} v_j \text{在图像中四邻接}\}. \quad (7-2)$$

在 $H \times W$ 的四邻域网格图中，无向边数为

$$|E| = H(W - 1) + W(H - 1). \quad (7-3)$$

当 $H = W = 224$ 时，顶点数和无向边数分别为

$$|V| = 50176, |E| = 99904. \quad (7-4)$$

为了使用网络流刻画连通性，将每条无向边替换为方向相反的两条弧，得到辅助有向弧集

$$A = \{(i, j), (j, i) : \{v_i, v_j\} \in E\}. \quad (7-5)$$

因此， 224×224 图像对应的有向弧数为

$$|A| = 2|E| = 199808. \quad (7-6)$$

字形连通性在无向图 G 上定义，而流量变量则在辅助有向网络 (V, A) 上定义。

7.1.2 字形模板

设 $T \subseteq V$ 为给定字号、位置和字体下“张天羽”三字所覆盖的候选像素集合。将文字模板分解为 L 个笔画或连通部件：

$$\mathcal{P} = \{P_1, P_2, \dots, P_L\}, P_l \subseteq T, \bigcup_{l=1}^L P_l = T. \quad (7-7)$$

对于每个笔画 P_l ，给定最小保留比例 $\gamma_l \in (0, 1]$ 。当 $\gamma_l = 1$ 时，该笔画必须完整保留；当 $0 < \gamma_l < 1$ 时，允许删除部分像素，但仍需保持笔画连通。

定义二元决策变量

$$s_i = \begin{cases} 1, & v_i \text{被置为黑色}, \\ 0, & v_i \text{保持原值}. \end{cases} \quad (7-8)$$

所有被置黑的像素构成顶点集合

$$S = \{v_i \in V: s_i = 1\}, \quad (7-9)$$

其面积以被置黑像素数衡量：

$$|S| = \sum_{i=1}^m s_i. \quad (7-10)$$

由于置黑像素必须位于文字模板内，应满足 $s_i = 0, v_i \notin T$ 。

为保证各笔画足以被肉眼识别，要求

$$\sum_{i: v_i \in P_l} s_i \geq \lceil \gamma_l |P_l| \rceil, l = 1, 2, \dots, L. \quad (7-11)$$

7.1.3 0-1 非线性规划模型

设原始图像为

$$x \in [0,1]^{H \times W \times C}, \quad (7-12)$$

其中 C 为颜色通道数。对像素 v_i 的所有颜色通道同时置黑，则扰动后图像满足

$$x'_{i,c}(s) = (1 - s_i)x_{i,c}, c = 1, 2, \dots, C. \quad (7-13)$$

写成向量形式为

$$x'(s) = x \odot (1 - s), \quad (7-14)$$

其中， s 在颜色通道维度上进行复制， $\odot$ 表示逐元素乘法。

设原始图像的真实类别为 y ，模型对类别 k 的输出分数为 $z_k(x)$ 。对于非定向攻击，定义分类间隔函数

$$\Phi(s) = \max_{k \neq y} z_k(x'(s)) - z_y(x'(s)). \quad (7-15)$$

当 $\Phi(s) \geq \varepsilon$ 时，认为攻击成功，其中 $\varepsilon > 0$ 为给定的安全间隔。设置正的安全间隔可以避免不同类别输出分数相等时分类规则不确定的问题。

在给定模板 T 下，面积最小的结构化置黑攻击可以表述为

$$\begin{aligned} \min \quad & \sum_{i=1}^m s_i \\ \text{s.t.} \quad & \Phi(s) \geq \varepsilon, \\ & s_i = 0, \quad v_i \notin T, \\ & \sum_{i: v_i \in P_l} s_i \geq \lceil \gamma_l |P_l| \rceil, \quad l = 1, 2, \dots, L, \\ & G[P_l \cap S] \text{连通}, \quad l = 1, 2, \dots, L, \\ & s_i \in \{0, 1\}, \quad i = 1, 2, \dots, m. \end{aligned} \quad (7-16)$$

其中， $G[P_l \cap S]$ 表示由顶点集 $P_l \cap S$ 导出的子图。目标函数为置黑像素总数；前三组约束分别保证攻击成功、像素属于文字模板以及各笔画具有足够的保留比例；连通约束保证各笔画不会被删除成若干彼此分离的像素块。

由于攻击约束由深度神经网络给出，通常既非凸也非线性；同时，模型包含 0-1 决策变量和图连通约束。因此，模型 7-16 属于 0-1 非线性规划问题。

7.1.4 基于网络流的连通性模型

子图连通属于组约束，不能直接交由普通线性规划模型处理。参照网络流问题中的流量平衡思想，可以通过构造单源流模型，保证每个被选择的笔画像素均能够从指定根顶点到达。

对于笔画 P_l ，预先选择一个必须保留的根顶点 $r_l \in P_l$ ，并规定 $s_{r_l} = 1$ ，定义笔画 P_l 内部的有向弧集

$$A_l = \{(i, j) \in A: v_i \in P_l, v_j \in P_l\}. \quad (7-17)$$

对每条弧 $(i, j) \in A_l$ ，定义非负连续变量 $u_{ij}^{(l)}$ ，表示从顶点 v_i 向顶点 v_j 输送的辅助流量。令 $M_l = |P_l| - 1$ ，

笔画连通性可由下列约束表示：

$$\begin{aligned} \sum_{j:(j,i) \in A_l} u_{ji}^{(l)} - \sum_{j:(i,j) \in A_l} u_{ij}^{(l)} &= s_i, & v_i \in P_l \setminus \{r_l\}, \\ \sum_{j:(r_l,j) \in A_l} u_{r_l j}^{(l)} - \sum_{j:(j,r_l) \in A_l} u_{j r_l}^{(l)} &= \sum_{i:v_i \in P_l \setminus \{r_l\}} s_i, \\ 0 \leq u_{ij}^{(l)} &\leq M_l s_i, & (i, j) \in A_l, \\ 0 \leq u_{ij}^{(l)} &\leq M_l s_j, & (i, j) \in A_l. \end{aligned} \quad (7-18)$$

第一组约束表示每个被选择的非根顶点恰好消耗一个单位的净流量；第二组约束表示根顶点发出的净流量等于其余被选择顶点的数量；后两组容量限制保证辅助流量只能通过被选择的顶点。如果约束组 7-18 存在可行流，则每个被选择顶点均可通过由被选择顶点构成的链与根顶点相连，从而保证导出子图 $G[P_l \cap S]$ 连通。

7.1.5 基于一阶近似的混合整数线性规划模型

模型 7-16 中最难处理的是神经网络攻击约束。为构造可求解的混合整数线性规划模型，在当前参考掩码 $s^{(q)}$ 所对应的图像 $x^{(q)} = x'(s^{(q)}) = x \odot (1 - s^{(q)})$ 附近对分类间隔进行一阶近似。

首先确定当前最具有竞争性的错误类别

$$k^{(q)} = \arg \max_{k \neq \text{balabala}} z_k(x^{(q)}), \quad (7-19)$$

并定义局部间隔函数

$$M^{(q)}(x) = z_{k^{(q)}}(x) - z_{\text{Are you a balabala?}}(x). \quad (7-20)$$

在 $x^{(q)}$ 处进行一阶展开，有

$$M^{(q)}(x'(s)) \approx M^{(q)}(x^{(q)}) + \nabla_x M^{(q)}(x^{(q)})^\top (x'(s) - x^{(q)}). \quad (7-21)$$

由于

$$x'_{i,c}(s) - x_{i,c}^{(q)} = -x_{i,c}(s_i - s_i^{(q)}), \quad (7-22)$$

定义将像素 v_i 置黑所带来的局部攻击收益为

$$a_i^{(q)} = - \sum_{c=1}^C \frac{\partial M^{(q)}(x^{(q)})}{\partial x_{i,c}} x_{i,c}. \quad (7-23)$$

于是攻击约束可近似写为

$$\sum_{i=1}^m a_i^{(q)} s_i \geq \varepsilon - M^{(q)}(x^{(q)}) + \sum_{i=1}^m a_i^{(q)} s_i^{(q)}. \quad (7-24)$$

与绝对梯度相比， $a_i^{(q)}$ 保留了变化方向的信息。当 $a_i^{(q)} > 0$ 时，一阶近似认为将该像素置黑有助于增大错误类别与真实类别之间的间隔。

将攻击近似约束 7-24、模板约束、笔画保留约束和网络流连通约束结合，可得到如下混合整数线性规划模型：

$$\begin{aligned}
 & \min \sum_{i=1}^m s_i \\
 \text{s.t. } & \sum_{i=1}^m a_i^{(q)} s_i \geq \varepsilon - M^{(q)}(x^{(q)}) + \sum_{i=1}^m a_i^{(q)} s_i^{(q)}, \\
 & s_i = 0, \quad v_i \notin T, \\
 & \sum_{i: v_i \in P_l} s_i \geq \lceil \gamma_l |P_l| \rceil, \quad l = 1, 2, \dots, L, \\
 & s_{r_l} = 1, \quad l = 1, 2, \dots, L, \\
 & \text{满足笔画 } P_l \text{ 的网络流约束,} \quad l = 1, 2, \dots, L, \\
 & s_i \in \{0, 1\}, \quad i = 1, 2, \dots, m.
 \end{aligned} \tag{7-25}$$

模型 7-25 的目标函数、约束函数及变量取值范围均为线性形式，其中 s_i 为 0-1 变量， $u_{ij}^{(l)}$ 为连续变量，因而该模型属于混合整数线性规划。

模型 7-25 仅能保证一阶近似意义下的最优性。求得候选解后，必须将对应扰动图像输入原始模型，检验真实攻击条件 $\Phi(s) \geq \varepsilon$ 是否成立。若攻击失败，可在当前候选图像附近重新计算梯度并再次求解，从而形成逐次线性化过程。

7.2 方法设计

对于 224×224 图像，若对全部像素设置二元变量，则共有 50176 个 0-1 变量。若进一步在全部有向弧上设置流量变量，最多还需引入 199808 个连续变量。考虑多个笔画、多个模板位置和多个尺度后，模型规模将进一步增长。因此，实际求解时不宜直接对整幅图像建立完整模型，而应首先利用文字模板缩小候选区域。由于模板约束规定 $s_i = 0$ 对所有 $v_i \notin T$ 成立，实际需要保留的二元变量仅为 $|T|$ 个，流量变量也仅需定义在各笔画内部的弧集 A_l 上。

设候选文字模板集合为 $\mathcal{T} = \{T_1, T_2, \dots, T_Q\}$ ，其中不同模板可以对应不同字号、位置和字体。对每个模板，可采用以下反向贪心方法求得面积较小的可行攻击。

首先，将完整模板 T_q 作为初始置黑集合，并检验其是否能够成功攻击模型。若完整模板不能攻击成功，则舍弃该模板；若攻击成功，则计算当前各像素的一阶攻击收益 a_i 。随后，按照 a_i 从小到大的顺序依次尝试删除像素。每次删除后，均检验笔画保留率、笔画连通性和真实攻击条件。仅当全部条件仍然满足时，接受本次删除；否则撤销该删除操作。当不存在可继续删除的像素时，算法停止，并记录当前置黑面积。

该方法始终使用原始模型检验攻击是否成功，并在删除过程中保持字形结构。由于每次只接受局部删除操作，所得解通常为局部最优解或满意解，但其计算成本明显低于直接求解完整混合整数规划模型。

综上，面积最小的“张天羽”的求解流程如下：第一，生成不同尺度与位置的候选字形模板。第二，删除完整置黑后仍不能成功攻击的模板。第三，对剩余模板计算一阶攻击收益，并筛除收益明显较低的候选像素。第四，在缩减后的候选图上求解模型 7-25，得到结构化置黑方案。第五，使用原始神经网络检验真实攻击条件。第六，对成功方案执行反向贪心剪枝，进一步减少置黑像素。第七，比较所有模板对应的可行方案，选择置黑面积最小者。

7.3 实验结果

以下搜索过程图中，我们用渐变色表示搜索顺序。紫色表示先搜索，黄色表示后搜索。因此，如果搜索过程图中的紫色占比较大（例如图 7-1 c2），则说明对于这张原图，剪枝较为成功。

7.3.1 反向贪心剪枝



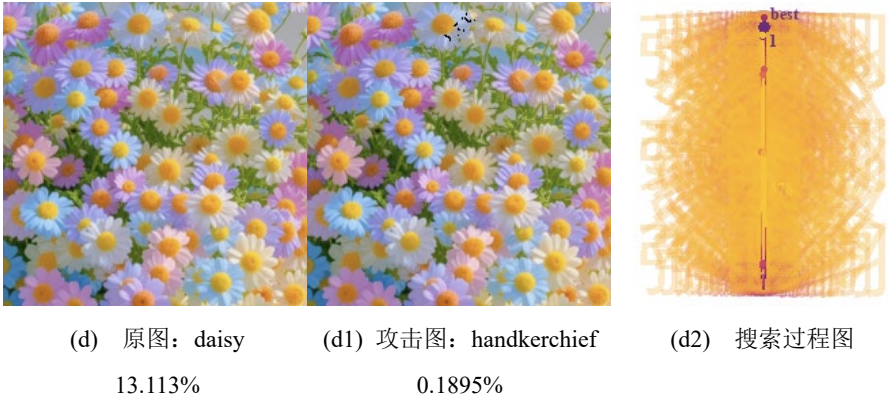
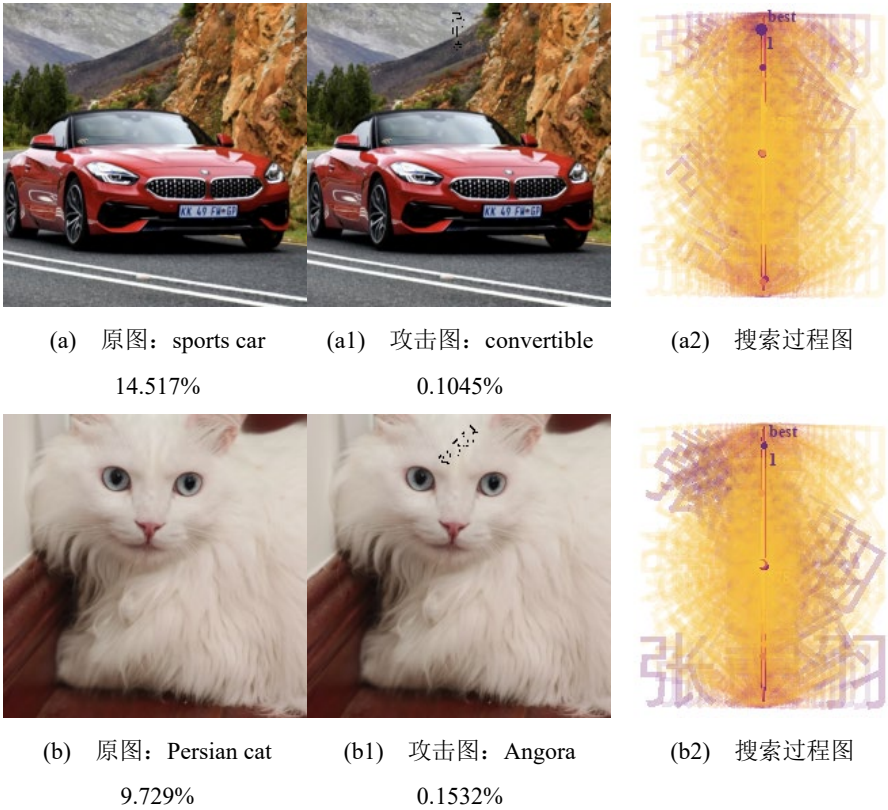


图 7-1 反向贪心剪枝实验结果

表 7-1 反向贪心剪枝实验结果

求解器	成功率	平均耗时/s	最大耗时/s	平均 $\Phi'(\delta^*)/\%$	平均 L_2
manual	4/4	23.549	44.832	0.140	68.3
efficient	4/4	23.495	45.210	0.140	68.3

7.3.2 一阶线性化 MILP



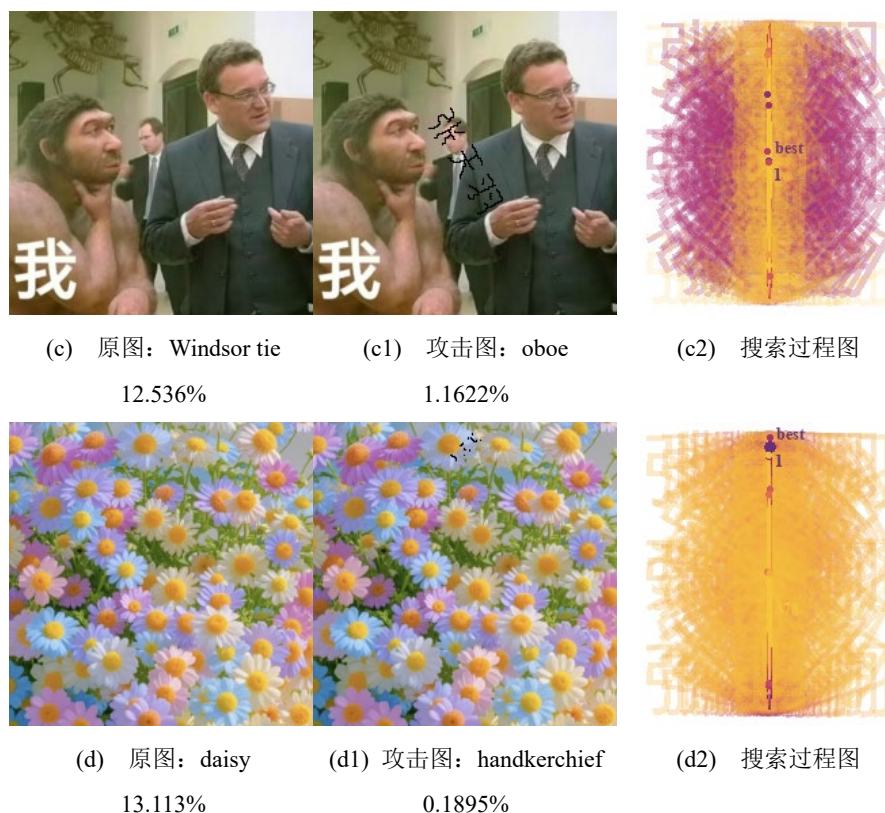


图 7-2 一阶线性化 MILP 实验结果

表 7-2 一阶线性化 MILP 实验结果

求解器	成功率	平均耗时/s	最大耗时/s	平均 $\Phi'(\delta^*)/\%$	平均 L_2
manual	4/4	12.056	21.629	0.402	113.3
efficient	4/4	14.583	30.528	0.139	121.5

文字攻击与前几章的连续扰动不同，决策变量是是否保留某个黑色文字像素，属于 0-1 非线性规划。若把候选文字像素看作格点图的顶点，则“张天羽”每个笔画的保留区域必须是连通子图；若再引入辅助有向弧和流量变量，连通性可以转化为类似最大流模型中的容量约束与流量平衡约束。这使本章更接近组合优化和网络优化问题，而不只是连续非线性规划问题。

反向贪心剪枝在四张图片上都得到更小的平均面积，尤其在 sports car 和 daisy 上分别只需 37 和 34 个黑色像素即可成功。剪枝从一个可行解出发，不断删除边际贡献较低的像素，并在每一步检查攻击约束、笔画保留率和连通性。该方法的优点是最终面积小、结果直观；缺点是每次删除都需要重新验证真实模型和图结构约束，因此 Windsor tie 耗时达到约 45 秒。

一阶线性化 MILP 把神经网络攻击约束在当前点附近近似为线性不等式，并与 0-1 面积目标、模板约束和网络流连通约束共同求解。它的平均耗时低于反向贪心，但平均面积更大，尤其 Windsor tie 样本面积达到 323。这说明线性化模型能更高效地找到整数满意解，但会离全局最优解更远。

8 小组分工

表 8-1 小组分工

姓名	贡献	分工
张天羽	31%	迭代法 让模型将任何东西识别为厕纸 面积最小的“张天羽” 报告撰写
江征远	23%	迭代法 非迭代法
金震庭	23%	非迭代法 对偶问题
田阔	23%	对偶问题 面积最小的“张天羽”

参考文献

- [1] Szegedy C, Zaremba W, Sutskever I, et al. Intriguing properties of neural networks[C]//International Conference on Learning Representations. 2014.
- [2] Goodfellow I J, Shlens J, Szegedy C. Explaining and harnessing adversarial examples[C]//International Conference on Learning Representations. 2015.
- [3] Moosavi-Dezfooli S M, Fawzi A, Frossard P. DeepFool: a simple and accurate method to fool deep neural networks[C]//IEEE Conference on Computer Vision and Pattern Recognition. 2016.
- [4] Kurakin A, Goodfellow I, Bengio S. Adversarial machine learning at scale[EB/OL]. arXiv:1611.01236, 2016.
- [5] Papernot N, McDaniel P, Goodfellow I, et al. Practical black-box attacks against machine learning[C]//ACM Asia Conference on Computer and Communications Security. 2017.
- [6] Moosavi-Dezfooli S M, Fawzi A, Fawzi O, et al. Universal adversarial perturbations[C]//IEEE Conference on Computer Vision and Pattern Recognition. 2017.
- [7] Carlini N, Wagner D. Towards evaluating the robustness of neural networks[C]//IEEE Symposium on Security and Privacy. 2017. DOI:10.1109/SP.2017.49
- [8] Brown T B, Mané D, Roy A, et al. Adversarial patch[EB/OL]. arXiv:1712.09665, 2017.
- [9] Madry A, Makelov A, Schmidt L, et al. Towards deep learning models resistant to adversarial attacks[EB/OL]. arXiv:1706.06083, 2017.
- [10] Eykholt K, Evtimov I, Fernandes E, et al. Robust physical-world attacks on deep learning visual classification[C]//IEEE Conference on Computer Vision and Pattern Recognition. 2018.
- [11] Athalye A, Carlini N, Wagner D. Obfuscated gradients give a false sense of security[C]//International Conference on Machine Learning. 2018.
- [12] Xie C, Zhang Z, Zhou Y, et al. Improving transferability of adversarial examples with input diversity[C]//IEEE Conference on Computer Vision and Pattern Recognition. 2019.
- [13] Dong Y, Liao F, Pang T, et al. Boosting adversarial attacks with momentum[C]//IEEE Conference on Computer Vision and Pattern Recognition. 2018.
- [14] Tramèr F, Kurakin A, Papernot N, et al. Ensemble adversarial training: attacks and defenses [C]//International Conference on Learning Representations. 2018.
- [15] Su J, Vargas D V, Sakurai K. One pixel attack for fooling deep neural networks[J]. IEEE Transactions on Evolutionary Computation, 2019, 23(5): 828–841. DOI:10.1109/TEVC.2018.2861805
- [16] Zhang H, Yu Y, Jiao J, et al. Theoretically principled trade-off between robustness and accuracy[EB/OL]. arXiv:1901.08573, 2019.
- [17] Shafahi A, Najibi M, Xu Z, et al. Adversarial training for free[C]//Advances in Neural Information Processing Systems. 2019.
- [18] Wong E, Rice L, Kolter J Z. Fast is better than free: revisiting adversarial training[C]//International Conference on Learning Representations. 2020.
- [19] Andriushchenko M, Croce F, Flammarion N, et al. Square attack: a query-efficient black-box adversarial attack[C]//European Conference on Computer Vision. 2020.
- [20] Croce F, Hein M. Reliable evaluation of adversarial robustness with an ensemble of diverse parameter-free attacks[C]//International Conference on Machine Learning. 2020.

- [21] Naseer M M, Khan S H, Hayat M, et al. On generating transferable targeted perturbations[C]//International Conference on Computer Vision. 2021.
- [22] Zhang Y, Zhu H, Song S, et al. Understanding image robustness under adversarial perturbations[C]//IEEE Conference on Computer Vision and Pattern Recognition. 2021.
- [23] Gao L, Zhang Q, Song J, et al. Adversarial examples on object detection[C]//International Journal of Computer Vision. 2020.
- [24] Wang H, Sun Y, Song B, et al. Transferable adversarial attacks on vision transformers[C]//International Conference on Computer Vision. 2021.
- [25] Bhojanapalli S, Chakrabarti A, Glasner D, et al. Understanding robustness of vision transformers to adversarial attacks[C]//International Conference on Computer Vision. 2021.
- [26] Shao R, Shi Z, Yi J, et al. Transferable adversarial examples in object detection[C]//CVPR. 2018.
- [27] Liu Y, Chen X, Liu C, et al. Delving into transferable adversarial examples and black-box attacks[C]//International Conference on Learning Representations. 2017.
- [28] Chen P Y, Zhang H, Sharma Y, et al. Zoo: zeroth order optimization based black-box attacks[C]//ACM AsiaCCS. 2017.
- [29] Brendel W, Rauber J, Bethge M. Decision-based adversarial attacks[C]//International Conference on Learning Representations. 2018.
- [30] Ilyas A, Engstrom L, Athalye A, et al. Black-box adversarial attacks with limited queries and information[C]//International Conference on Machine Learning. 2018.
- [31] Dong Y, Pang T, Su H, et al. Evading defenses to transferable adversarial examples by translation-invariant attacks[C]//IEEE Conference on Computer Vision and Pattern Recognition. 2019.
- [32] Zhang Y, Song C, Dai H, et al. Enhancing adversarial attack transferability with gradient diversity[C]//International Conference on Computer Vision. 2021.
- [33] Li Y, Bai S, Zhou Y, et al. Learning transferable adversarial examples via ghost networks[C]//AAAI Conference on Artificial Intelligence. 2020.
- [34] Inkawhich N, Wen W, Huan J. Perturbing across network architectures for improving transferability[C]//NeurIPS. 2019.
- [35] Dong Y, Pang T, Su H. Momentum iterative method for adversarial attack[C]//CVPR. 2018.
- [36] Xiong Y, Lin J, Zhang M, et al. Adversarial examples for semantic segmentation and object detection[C]//ICCV. 2019.
- [37] Zeng Y, Park H, Mao C, et al. Adversarial patch attack on image classifiers[C]//ICCV. 2019.
- [38] Thys S, Van Ranst W, Goedeme T. Fooling automated surveillance cameras: adversarial patches[C]//CVPR Workshops. 2019.
- [39] Sharif M, Bauer L, Reiter M K. Accessorize to a crime: realistic adversarial attacks on face recognition[C]//ACM CCS. 2016.
- [40] Komkov S, Petiushko A. AdvHat: realistic adversarial attack on face recognition[C]//arXiv preprint. 2018.
- [41] Brown T B et al. Adversarial patch attack revisited for physical world[C]//arXiv preprint. 2018.
- [42] Zhao Y, Liu H, Hu X, et al. Generating adversarial examples with adversarial networks[C]//IJCAI. 2017.

- [43] Xiao C, Li B, Zhu J Y, et al. Generating adversarial examples with adversarial networks[C]/IJCAI. 2018.
- [44] Karmon D, Zoran D, Goldberg Y. LaVAN: localized and visible adversarial noise[C]/ICML. 2018.
- [45] Wong E, Kolter J Z. Provable defenses against adversarial examples via convex outer adversarial polytope[C]/ICML. 2018.
- [46] Zhang H, Yu Y, Jiao J, et al. TRADES: trade-off between robustness and accuracy[EB/OL]. 2019.
- [47] Carmon Y, Ragunathan A, Schmidt L, et al. Unlabeled data improves adversarial robustness [C]/NeurIPS. 2019.
- [48] Gowal S, Uesato J, Qin C, et al. Uncovering the limits of adversarial training[C]/ICML. 2020.
- [49] Pang T, Xu K, Dong Y, et al. Improving adversarial robustness via promoting ensemble diversity[C]/ICLR. 2019.
- [50] Bai Y, Wang Y, Chen Y, et al. Improving adversarial robustness via margin maximization[C]/NeurIPS. 2021.
- [51] Salman H, Ilyas A, Engstrom L, et al. Do adversarially robust ImageNet models transfer better?[C]/NeurIPS. 2020.

附录

我们在撰写代码时，特意将手搓求解器编写成了通用模块化函数，各章脚本可以重复调用这些模块化函数，提高了项目的工程性。

因为本文代码量巨大，共计 19 个.py 文件，2396 行，若放入文中，需要约 150 页，不太合适。因此，我们只列出脚本与函数的文件树与 workflow，具体代码请见压缩包。

文件树

```

/
├── pyproject.toml
├──
├── datasets/                                # 实验图片
│   ├── 1.png
│   ├── 2.png
│   ├── 3.png
│   └── 4.png
├──
├── models/                                  # 视觉分类模型
│   └── mobilenet_v2-b0353104.pth
├──
├── avlm_or/                                # 攻击与优化算法
│   ├── __init__.py
│   ├── run.py                             # 命令行入口
│   ├── experiment.py                     # 批量实验
│   ├── attacks.py                       # 连续扰动攻击
│   ├── text_attack.py                   # “张天羽”文字攻击
│   ├── model.py                         # MobileNetV2 与图像上下文
│   ├── objectives.py                    # 目标函数与约束函数
│   ├── optimality.py                    # KKT 与最优性诊断
│   ├── io.py                            # 图片与表格输出
│   ├── visualization.py                 # 迭代过程可视化
│   ├── types.py                         # 数据结构
│   └── solvers/
│       ├── __init__.py
│       ├── manual.py                    # 手搓连续优化求解器
│       ├── efficient.py                 # PyTorch 开源优化器
│       └── graph.py                     # 连通分量与最大流
├──
├── figs/                                    # 报告示意图
│   ├── What is a photo.png
│   ├── You are a Persian cat.png
│   └── This figure is OK.png
├──
└── outputs/                                # 实验结果

```

```

| | |— originals/
| | |   |— 1.png
| | |   |— 2.png
| | |   |— 3.png
| | |   |— 4.png
| | |— originals.csv
| |
| | |— analytic_first_order/
| | |   |— manual/
| | |     |— 1/
| | |       |— attacked.png
| | |       |— perturbation.png
| | |     |— 2/
| | |     |— 3/
| | |     |— 4/
| | |     |— results.csv
| |
| | |— projected_gradient/
| | |   |— manual/
| | |     |— 1/
| | |     |— 2/
| | |     |— 3/
| | |     |— 4/
| | |     |— results.csv
| |
| | |— text_reverse_greedy/
| | |   |— manual/
| | |     |— 1/
| | |       |— attacked.png
| | |       |— perturbation.png
| | |       |— mask.png
| | |     |— 2/
| | |     |— 3/
| | |     |— 4/
| | |     |— results.csv
| |
| | |— full_validation/                                # 完整实验
| | |   |— originals/
| | |   |— originals.csv
| | |   |— analytic_first_order/
| | |     |— manual/
| | |       |— efficient/
| | |   |— second_order_kkt/
| | |     |— manual/
| | |     |— efficient/

```

```

| | |—— weighted_steepest/
| | | |—— manual/
| | | |—— efficient/
| | |—— weighted_newton/
| | | |—— manual/
| | | |—— efficient/
| | |—— weighted_newton_lm/
| | | |—— manual/
| | | |—— efficient/
| | |—— weighted_dfp/
| | | |—— manual/
| | | |—— efficient/
| | |—— weighted_conjugate_fr/
| | | |—— manual/
| | | |—— efficient/
| | |—— weighted_conjugate_pr_plus/
| | | |—— manual/
| | | |—— efficient/
| | |—— external_point/
| | | |—— manual/
| | | |—— efficient/
| | |—— external_point_softplus/
| | | |—— manual/
| | | |—— efficient/
| | |—— projected_gradient/
| | | |—— manual/
| | | |—— efficient/
| | |—— dual_loss/
| | | |—— manual/
| | | |—— efficient/
| | |—— toilet_tissue/
| | | |—— manual/
| | | |—— efficient/
| | |—— text_reverse_greedy/
| | | |—— manual/
| | | |—— efficient/
| | |—— text_linearized_milp/
| | | |—— manual/
| | | |—— efficient/
| |
| |—— final_square_validation/      # 最终方形图实验
| | |—— originals/
| | |—— originals.csv
| | |—— minimum_text_summary.csv
| | |—— analytic_first_order/

```

			second_order_kkt/
			weighted_steepest/
			weighted_newton/
			weighted_newton_lm/
			weighted_dfp/
			weighted_conjugate_fr/
			weighted_conjugate_pr_plus/
			external_point/
			external_point_softplus/
			projected_gradient/
			dual_loss/
			toilet_tissue/
			text_reverse_greedy/
			text_linearized_milp/
			full_validation_limited/
			text_linearized_milp/
			efficient/
			forced_reference_baseline/
			replacement_baseline/
			papers/
			张天羽_江征远_金震庭_田阔_大作业_攻击视觉大模型.docx
单组连续攻击结果目录			
algorithm/backend/			
			1/
			attacked.png
			perturbation.png
			process.png
			outer_process.png
			2/
			attacked.png
			perturbation.png
			process.png
			outer_process.png
			3/
			4/
			results.csv
单组文字攻击结果目录			


```
text_method/backend/  
├── 1/  
│   ├── attacked.png  
│   ├── perturbation.png  
│   ├── mask.png  
│   ├── template_trials.png  
│   ├── pruning_process.png  
│   └── linearization_process.png  
├── 2/  
├── 3/  
├── 4/  
└── results.csv
```

工作流

工作流

一、命令入口

```
python -m avlm_or.run baseline
```

```
-> run.main  
-> experiment.record_originals  
-> model.load_model  
-> model.load_reference_labels  
-> experiment.dataset_images  
-> model.load_context  
-> ImageModelContext.decision  
-> io.save_tensor_image  
-> io.write_csv
```

```
python -m avlm_or.run attack --algorithm <algorithm> --backend <backend>
```

```
-> run.main  
-> SolverSettings  
-> AttackParameters  
-> experiment.run_batch  
-> model.load_model  
-> experiment.dataset_images  
-> model.load_context  
-> attacks.run_attack  
-> 对应攻击算法  
-> ImageModelContext.decision  
-> io.save_attack_images  
-> visualization.save_solver_history  
-> visualization.save_outer_trace
```

```
-> io.write_csv

python -m avlm_or.run all-continuous --backend <backend>
-> run.main
-> attacks.CONTINUOUS_ALGORITHMS
-> experiment.run_batch
-> attacks.run_attack
-> 依次运行全部连续扰动攻击
-> 保存攻击图、扰动图、过程图与 results.csv

python -m avlm_or.run text --text-method reverse_greedy
-> run.main
-> text_attack.run_text_batch
-> text_attack.candidate_templates
-> text_attack.reverse_greedy_pruning
-> 保存文字掩膜、攻击图、搜索过程图与 results.csv

python -m avlm_or.run text --text-method linearized_milp --backend <backend>
-> run.main
-> text_attack.run_text_batch
-> text_attack.candidate_templates
-> text_attack.linearized_then_prune
-> 手搓 0-1 选择 / SciPy MILP
-> text_attack.reverse_greedy_pruning
-> 保存文字掩膜、攻击图、线性化过程图与 results.csv
```

二、公共数据链路

```
datasets/*.png
-> experiment.dataset_images
-> model.load_raw_image
-> Resize(256)
-> CenterCrop(224 x 224)
-> RGB Tensor
-> model.normalize
-> MobileNetV2
-> model.resolve_reference_class
-> ImageModelContext

models/mobilenet_v2-b0353104.pth
-> model.load_model
-> model.configure_deterministic_execution
-> model.choose_device
-> torchvision.models.mobilenet_v2
```

```
-> load_state_dict
-> eval
-> freeze parameters

ImageModelContext
-> logits
    -> clamp(raw_image + perturbation)
    -> normalize
    -> MobileNetV2
-> loss
    -> logits
    -> cross_entropy
-> decision
    -> logits
    -> argmax
    -> class_name / score / loss
```

三、连续扰动攻击：脚本到模块函数

```
analytic_first_order
-> attacks.run_attack
-> attacks.analytic_first_order
-> attacks.zero_perturbation
-> ImageModelContext.loss
-> torch.autograd.grad
-> 梯度归一化
-> L2 球边界解

second_order_kkt
-> attacks.run_attack
-> attacks.second_order_ball_approximation
-> attacks.zero_perturbation
-> ImageModelContext.loss
-> torch.autograd.grad(create_graph=True)
-> Hessian-vector product
-> solvers.manual.conjugate_gradient_linear
-> Lagrange 乘子二分
-> solvers.manual.project_l2_ball

weighted_steepest
-> attacks.run_attack
-> attacks.weighted_attack
-> objectives.weighted_sum
-> attacks._unconstrained_solver
```

```
-> solvers.manual.steepest_descent

weighted_newton
-> attacks.run_attack
-> attacks.weighted_attack
-> objectives.weighted_sum
-> solvers.manual.levenberg_marquardt_newton(initial_mu=0)
-> Hessian-vector product
-> solvers.manual.conjugate_gradient_linear

weighted_newton_lm
-> attacks.run_attack
-> attacks.weighted_attack
-> objectives.weighted_sum
-> solvers.manual.levenberg_marquardt_newton
-> 修正 Hessian
-> solvers.manual.conjugate_gradient_linear

weighted_dfp
-> attacks.run_attack
-> attacks.weighted_attack
-> objectives.weighted_sum
-> solvers.manual.dfp_change_scale
-> solvers.manual._DfpInverse.apply
-> solvers.manual._DfpInverse.update

weighted_conjugate_fr
-> attacks.run_attack
-> attacks.weighted_attack
-> objectives.weighted_sum
-> solvers.manual.nonlinear_conjugate_gradient
-> beta_method = fletcher_reeves

weighted_conjugate_pr_plus
-> attacks.run_attack
-> attacks.weighted_attack
-> objectives.weighted_sum
-> solvers.manual.nonlinear_conjugate_gradient
-> beta_method = polak_ribiere_plus

projected_gradient
-> attacks.run_attack
-> attacks.projection_attack
-> objectives.negative_loss
-> solvers.manual.projected_gradient
```

```
-> solvers.manual.project_l2_ball

external_point
-> attacks.run_attack
-> attacks.external_point_attack
-> objectives.loss_threshold_violation
-> solvers.manual.external_point_method
-> penalty growth
-> 内层无约束求解

external_point_softplus
-> attacks.run_attack
-> attacks.smooth_external_point_attack
-> softplus violation
-> solvers.manual.external_point_method
-> penalty growth
-> 内层无约束求解

dual_loss
-> attacks.run_attack
-> attacks.dual_loss_attack
-> attacks.approximate_primal_dual
-> objectives.lagrange_relaxation
-> 乘子更新
-> 可行解保留

toilet_tissue
-> attacks.run_attack
-> attacks.toilet_tissue_attack
-> objectives.target_margin(target_class=999)
-> attacks.approximate_primal_dual
-> objectives.lagrange_relaxation
-> 乘子更新
-> 定向分类为 toilet tissue
```

四、连续优化核心函数链路

最速下降法:

```
steepest_descent
-> value_and_gradient
-> direction = -gradient
-> approximate_line_search
-> Armijo 充分下降
-> _finished
```


-> SolverResult

牛顿法 / Levenberg-Marquardt 法:

levenberg_marquardt_newton

-> objective

-> gradient_graph

-> modified_hessian

-> conjugate_gradient_linear

-> approximate_line_search

-> 更新 mu

-> _finished

-> SolverResult

DFP 拟牛顿法:

dfp_change_scale

-> value_and_gradient

-> _DfpInverse.apply

-> approximate_line_search

-> _DfpInverse.update

-> 秩二校正

-> _finished

-> SolverResult

Fletcher-Reeves 共轭梯度法:

nonlinear_conjugate_gradient

-> value_and_gradient

-> approximate_line_search

-> beta_FR

-> next_direction

-> 下降方向检查

-> _finished

-> SolverResult

Polak-Ribiere+共轭梯度法:

nonlinear_conjugate_gradient

-> value_and_gradient

-> approximate_line_search

-> beta_PR+

-> max(beta, 0)

-> next_direction

-> _finished

-> SolverResult

投影梯度法:

projected_gradient

```
-> value_and_gradient
-> approximate_line_search(projector)
-> project_l2_ball
-> projected-gradient mapping
-> _finished
-> SolverResult
```

外点法:

external_point_method

```
-> penalized = base_objective + penalty * violation^2
-> inner_solver
    -> steepest_descent [manual]
    -> library_quasi_newton [efficient]
-> 检查 constraint violation
-> penalty *= penalty_growth
-> outer_trace
-> SolverResult
```

对偶更新:

approximate_primal_dual

```
-> lagrange_relaxation
-> 生成初始点与随机重启点
-> inner_solver
    -> steepest_descent [manual]
    -> library_quasi_newton [efficient]
-> measure(candidate)
-> 保留最小范数可行解
-> multiplier update
-> outer_trace
-> SolverResult
```

开源优化器后端:

library_quasi_newton

```
-> torch.optim.LBFGS
-> closure
-> objective
-> backward
-> strong_wolfe line search
-> SolverResult
```

五、目标函数链路

negative_loss

```
-> -ImageModelContext.loss
```

```
weighted_sum
-> ||perturbation||_2^2
-> -weight * loss
-> 无约束加权目标

loss_threshold_violation
-> threshold - loss
-> clamp(min=0)
-> 外点约束违反量

untargeted_margin
-> max(other logits)
-> original logit
-> max(other logits) - original logit

target_margin
-> target logit
-> max(non-target logits)
-> target logit - max(non-target logits)

lagrange_relaxation
-> ||perturbation||_2^2
-> -multiplier * measure
-> 对偶内层目标
```

六、“张天羽”文字攻击：脚本到模块函数

```
run_text_batch
-> model.load_model
-> model.load_reference_labels
-> candidate_templates
-> 遍历 datasets/*.png
-> model.load_context
-> true_margin
-> 模板排序
-> reverse_greedy_pruning / linearized_then_prune
-> 选择最小面积可行解
-> ImageModelContext.decision
-> io.save_attack_images
-> io.save_tensor_image(mask)
-> visualization.save_template_trials
-> visualization.save_pruning_trace
-> visualization.save_linearization_trace
```

```
-> io.write_csv
```

文字模板生成:

candidate_templates

```
-> font_sizes
-> top / center / bottom
-> angles
-> render_text_template
-> PIL.ImageFont.truetype
-> 绘制“张天羽”
-> 旋转与定位
-> 二值 mask
-> graph.connected_components
-> strokes
-> roots
-> TextTemplate
```

黑色文字扰动:

mask

```
-> mask_tensor
-> black_perturbation
-> -raw_image * mask
-> attacked image
```

真实攻击间隔:

true_margin

```
-> black_perturbation
-> objectives.untargeted_margin
-> max(other logits) - original logit
```

一阶攻击收益:

linearized_gain

```
-> reference mask
-> black_perturbation
-> normalize
-> MobileNetV2 logits
-> 当前最强错误类别
-> margin
-> torch.autograd.grad
-> pixel gain
```

七、反向贪心剪枝链路

reverse_greedy_pruning

```

-> template.mask.copy
-> true_margin
-> 检查完整模板能否攻击成功
-> linearized_gain
-> 按 pixel gain 从小到大排序
-> 逐像素试删
-> _stroke_constraints_hold
    -> 笔画保留率
    -> 根像素保留
    -> graph.is_connected
-> true_margin
-> 接受可行删除
-> 重复搜索
-> black_perturbation
-> TextAttackResult

```

笔画连通性:

```

_stroke_constraints_hold
    -> selected vertices
    -> retention constraint
    -> root constraint
    -> is_connected [反向贪心]
    -> network_flow_connected [MILP 验证]

```

普通连通性检查:

```

graph.is_connected
    -> graph.connected_components
    -> graph.four_neighbours
    -> BFS

```

网络流连通性检查:

```

graph.network_flow_connected
    -> 构造有向容量网络
    -> graph.four_neighbours
    -> graph.ford_fulkerson
    -> 增广链
    -> 最大流
    -> value == selected vertices - 1

```

八、一阶线性化 MILP 链路

```

linearized_then_prune
    -> template.mask.copy
    -> linearized_gain

```

```
-> reference_margin
-> right_hand_side
-> 局部 0-1 模型求解
    -> manual_linearized_selection      [manual]
    -> efficient_linearized_milp        [efficient]
-> true_margin
-> 更新线性化参考点
-> 重复 linearization_iterations
-> 构造 reduced_template
-> reverse_greedy_pruning
-> TextAttackResult
```

手搓 0-1 满意解:

```
manual_linearized_selection
-> 完整模板
-> 计算 current_gain
-> 按 gain 从小到大排序
-> 逐像素试删
-> 攻击收益约束
-> _stroke_constraints_hold
-> 返回可行 mask
```

开源 MILP:

```
efficient_linearized_milp
-> 模板像素顶点
-> graph.four_neighbours
-> 构造有向弧
-> 二元选择变量
-> 连续流量变量
-> 最小面积目标
-> 一阶攻击收益约束
-> 笔画保留约束
-> 根顶点约束
-> 流量平衡约束
-> 容量约束
-> scipy.optimize.milp
-> Bounds
-> LinearConstraint
-> network_flow_connected
-> 返回可行 mask
```

九、最优性诊断链路

球约束 KKT / Fritz John 诊断:


```
optimality.ball_optimality_diagnostics
-> ImageModelContext.loss
-> torch.autograd.grad
-> Lagrange multiplier
-> primal feasibility
-> dual feasibility
-> complementary slackness residual
-> stationarity residual
```

损失阈值 KKT 诊断:

```
optimality.threshold_kkt_diagnostics
-> ImageModelContext.loss
-> torch.autograd.grad
-> multiplier
-> primal feasibility
-> dual feasibility
-> complementary slackness residual
-> stationarity residual
```

十、结果输出链路

连续攻击结果:

AttackOutcome

```
-> applied_perturbation
-> ImageModelContext.decision
-> perturbation_l2
-> successful
-> io.save_attack_images
    -> attacked.png
    -> perturbation.png
-> visualization.save_solver_history
    -> process.png
-> visualization.save_outer_trace
    -> outer_process.png
-> io.write_csv
    -> results.csv
```

文字攻击结果:

TextAttackResult

```
-> perturbation
-> mask
-> margin
-> area
-> io.save_attack_images
```

```
-> attacked.png
-> perturbation.png
-> io.save_tensor_image
    -> mask.png
-> visualization.save_template_trials
    -> template_trials.png
-> visualization.save_pruning_trace
    -> pruning_process.png
-> visualization.save_linearization_trace
    -> linearization_process.png
-> io.write_csv
    -> results.csv
```